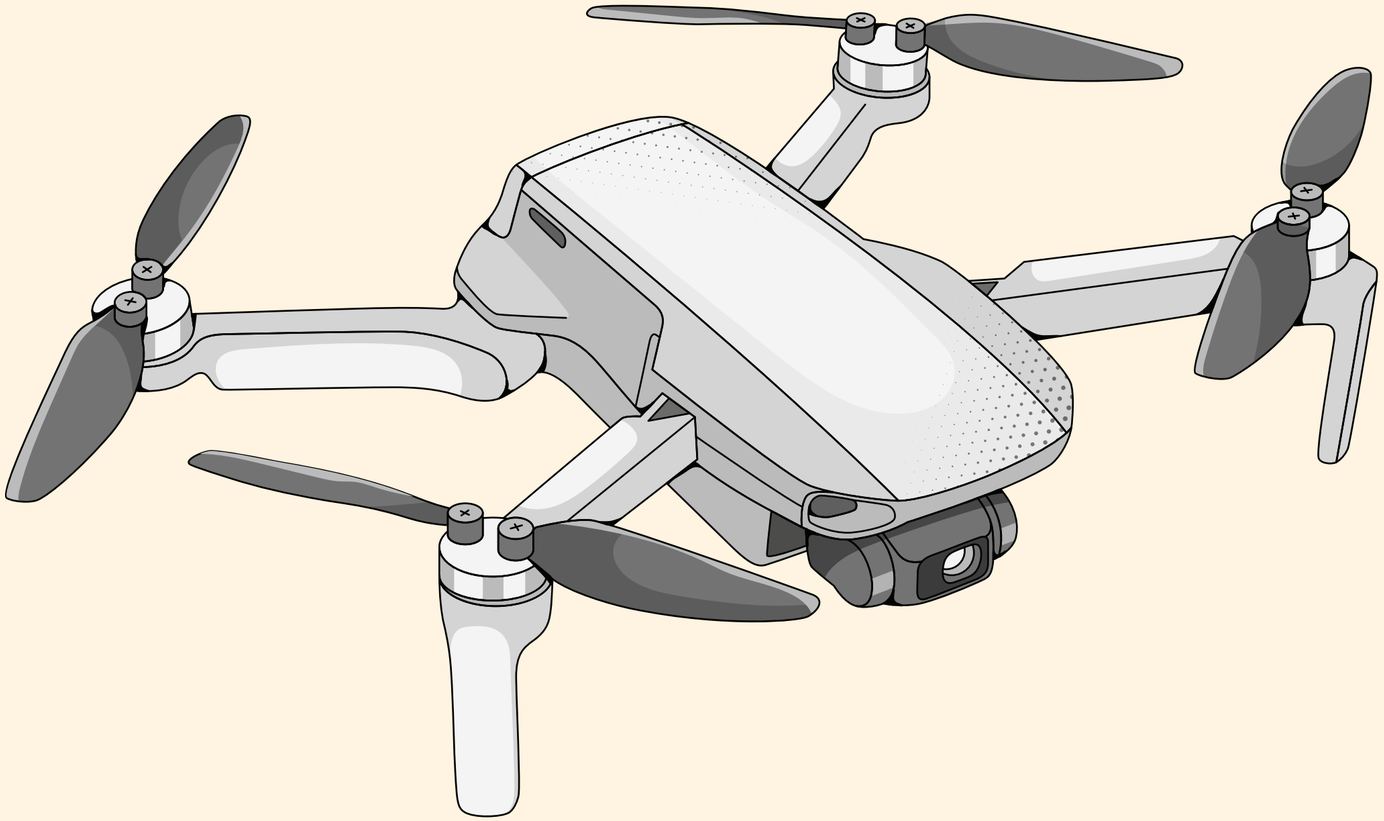
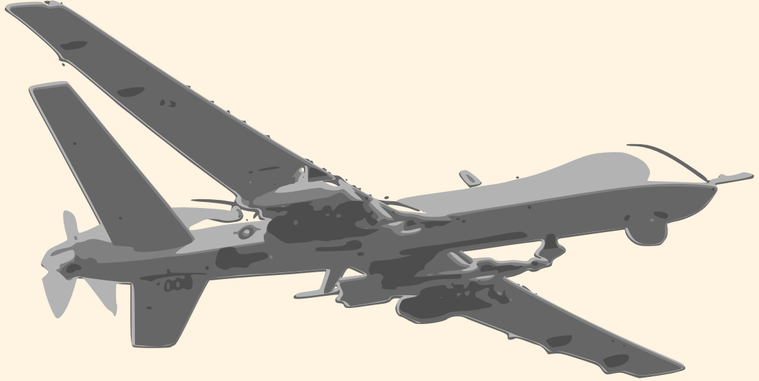
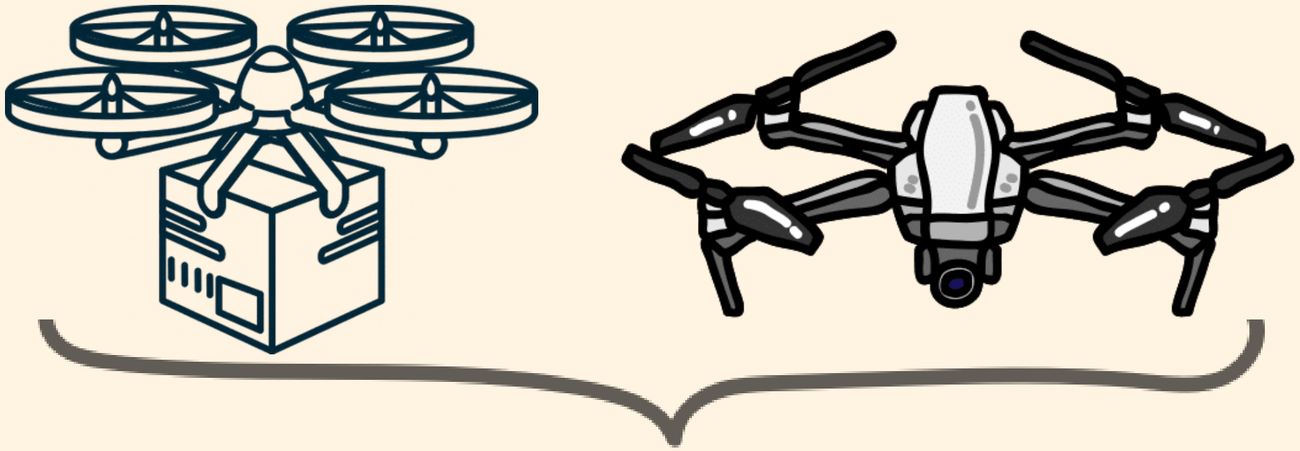
# STABILISATION DES DRONES PAR ASSERVISSEMENT



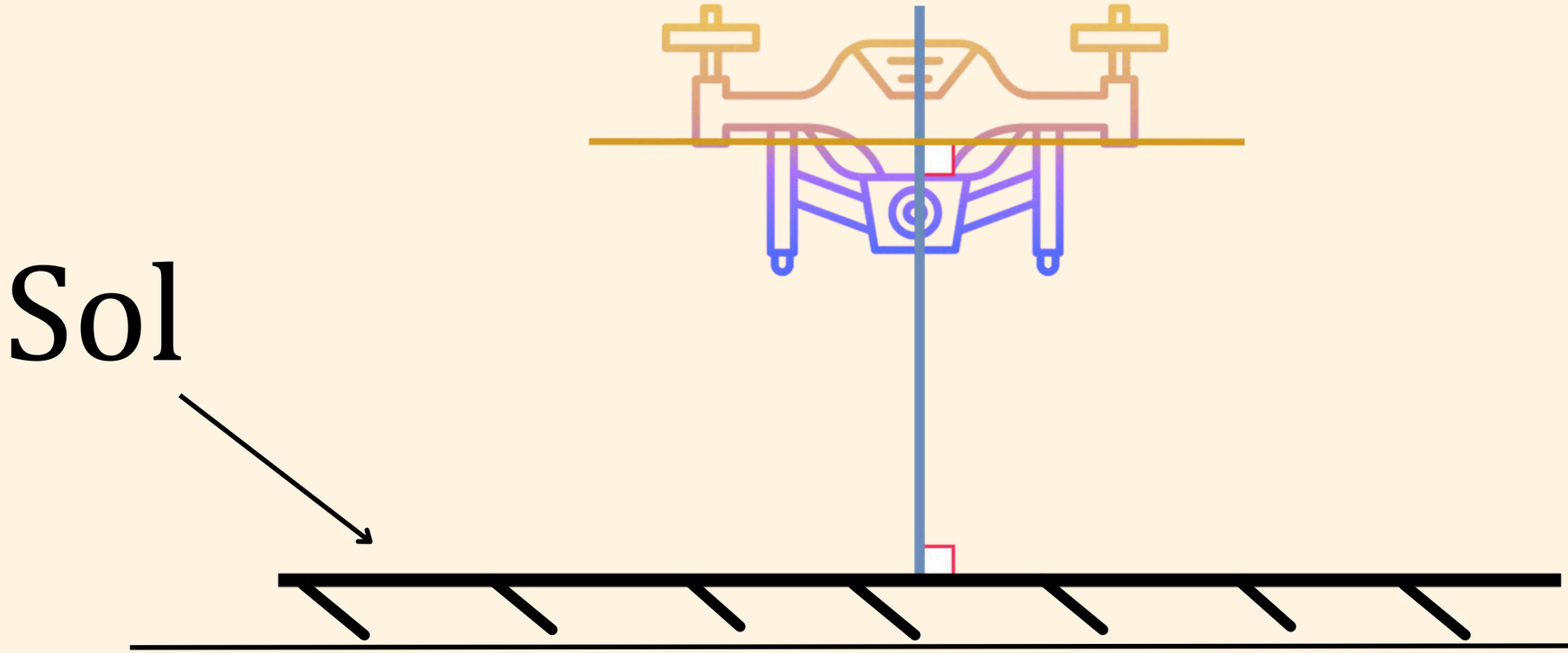
N°SCEI : 11991 BILAL BELKHAOUEL TSI 2025

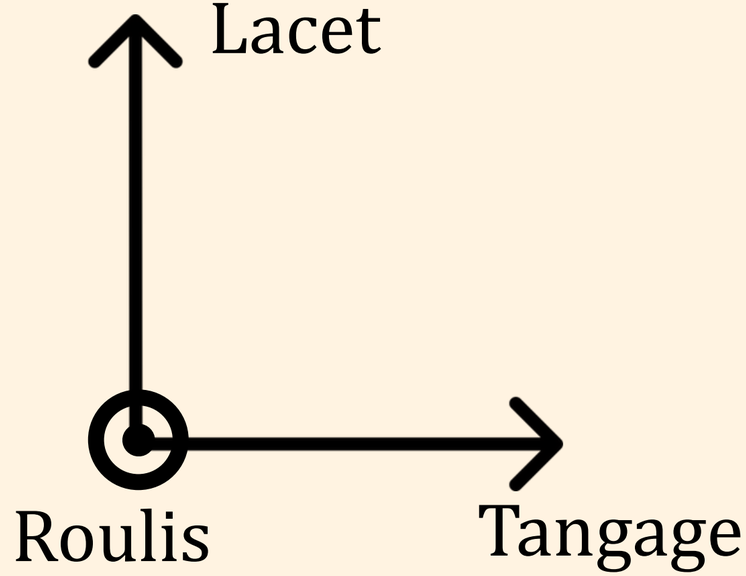


Drone M.A.L.E



Drone Quadri-rotor



# Sommaire

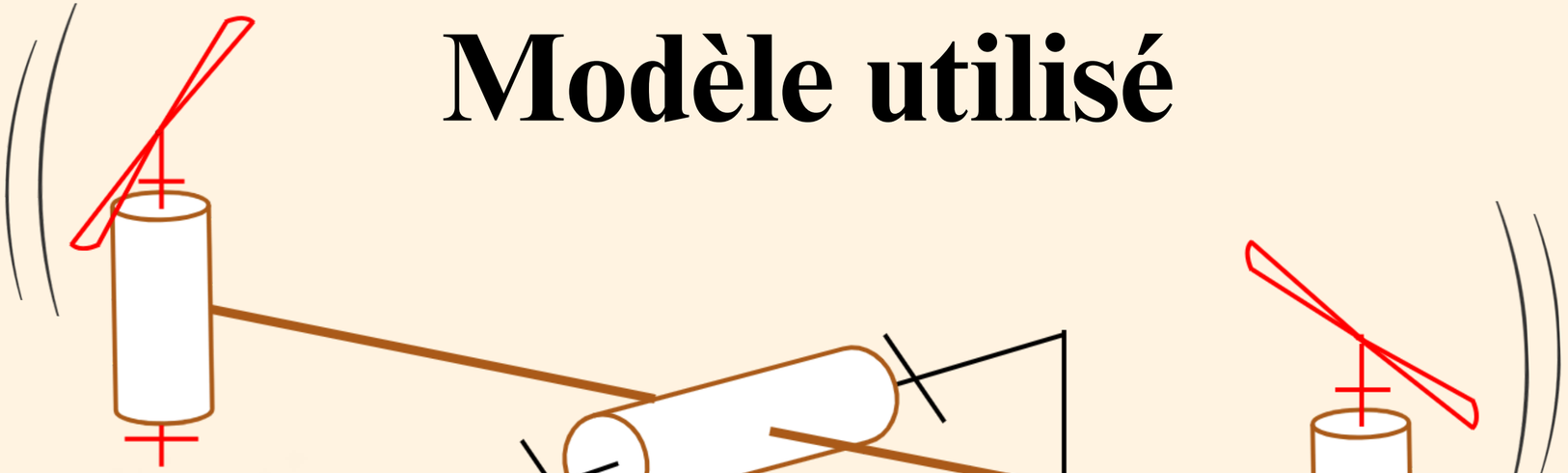
## I- Modélisation et analyse du système :

- Modèle étudié et boucle d’asservissement
- Mesure d’angle
- Filtrage
- Identification du modèle
- Correcteur

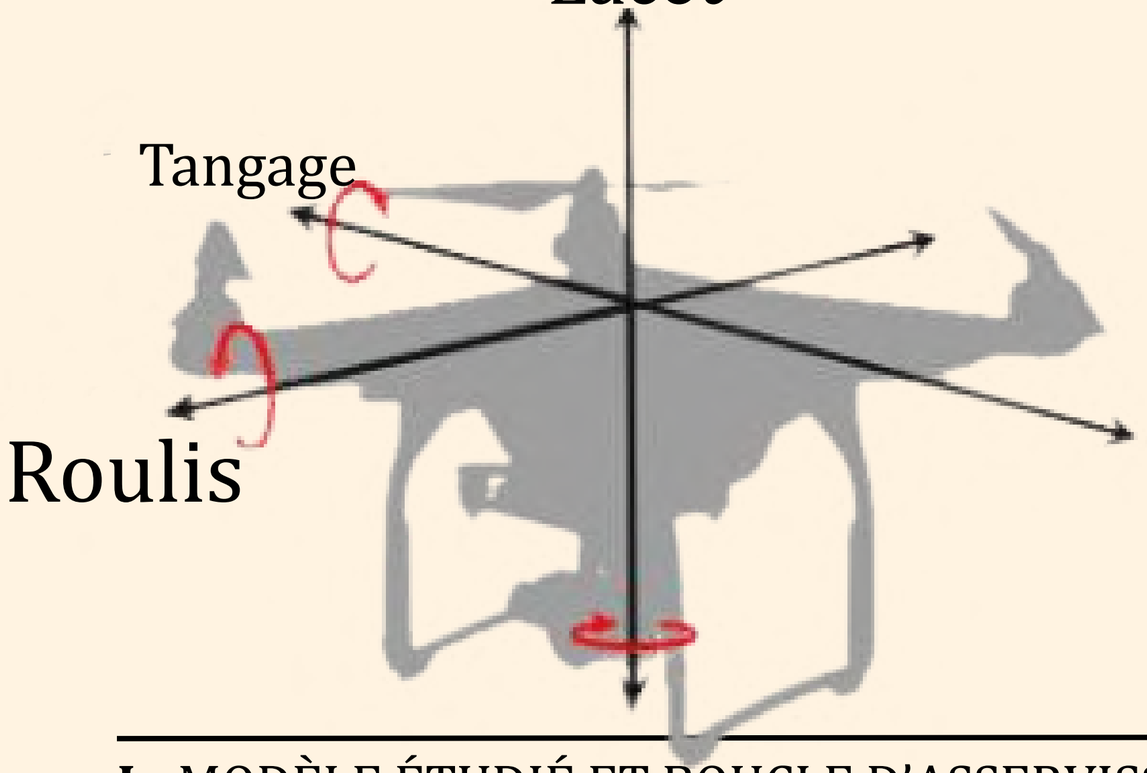
## II- Mise en œuvre et validation :

- Implémentation
- Comparaison avec la théorie

# Modèle utilisé



Lacet

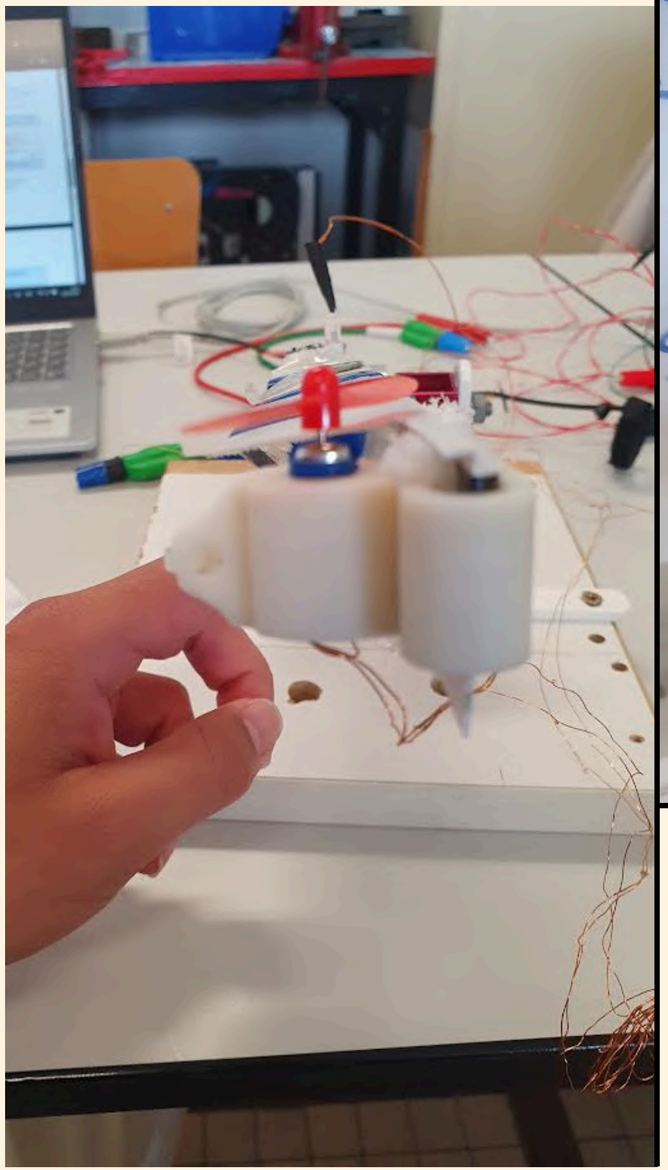


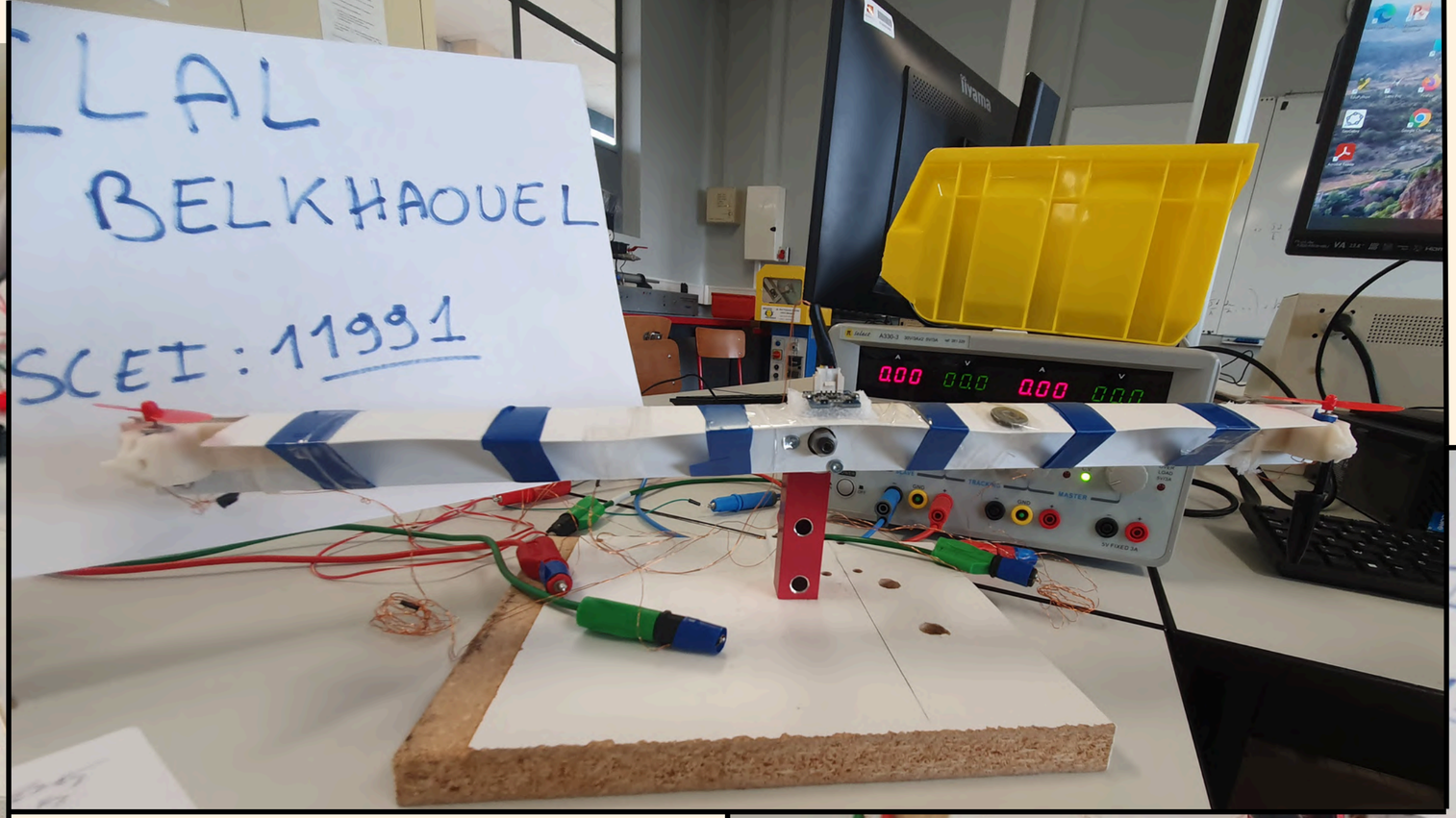
Tangage

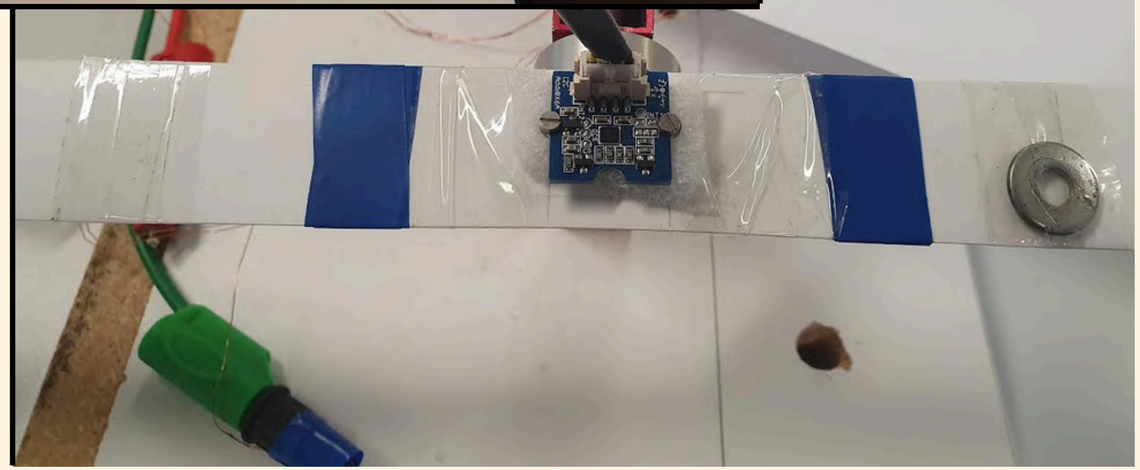
Roulis

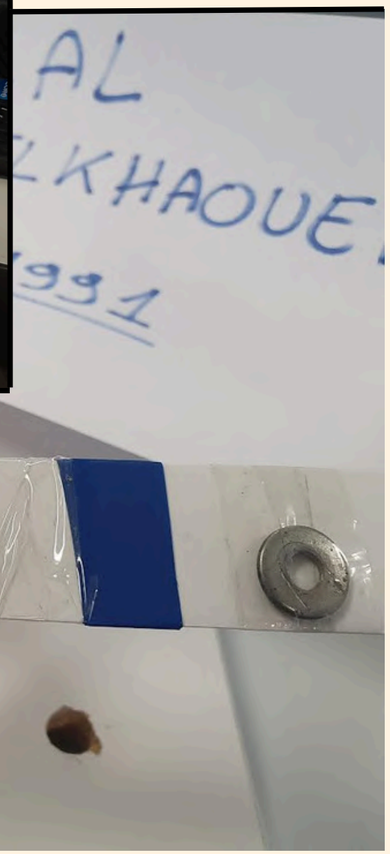
Bati

Moteur à courant continu

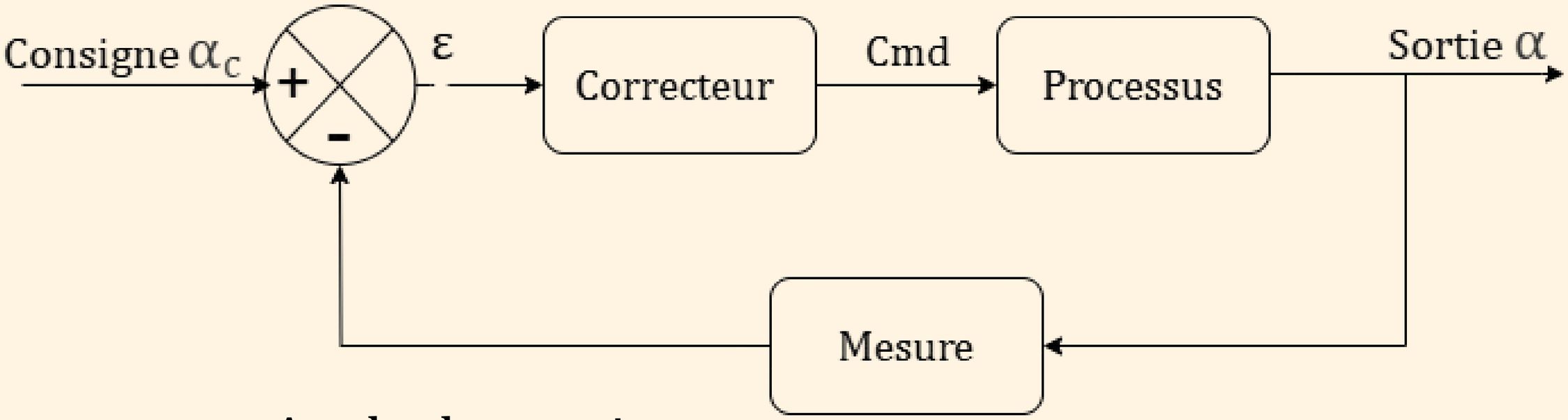



LAL
BELKHAOUEL
SCEI : 1991



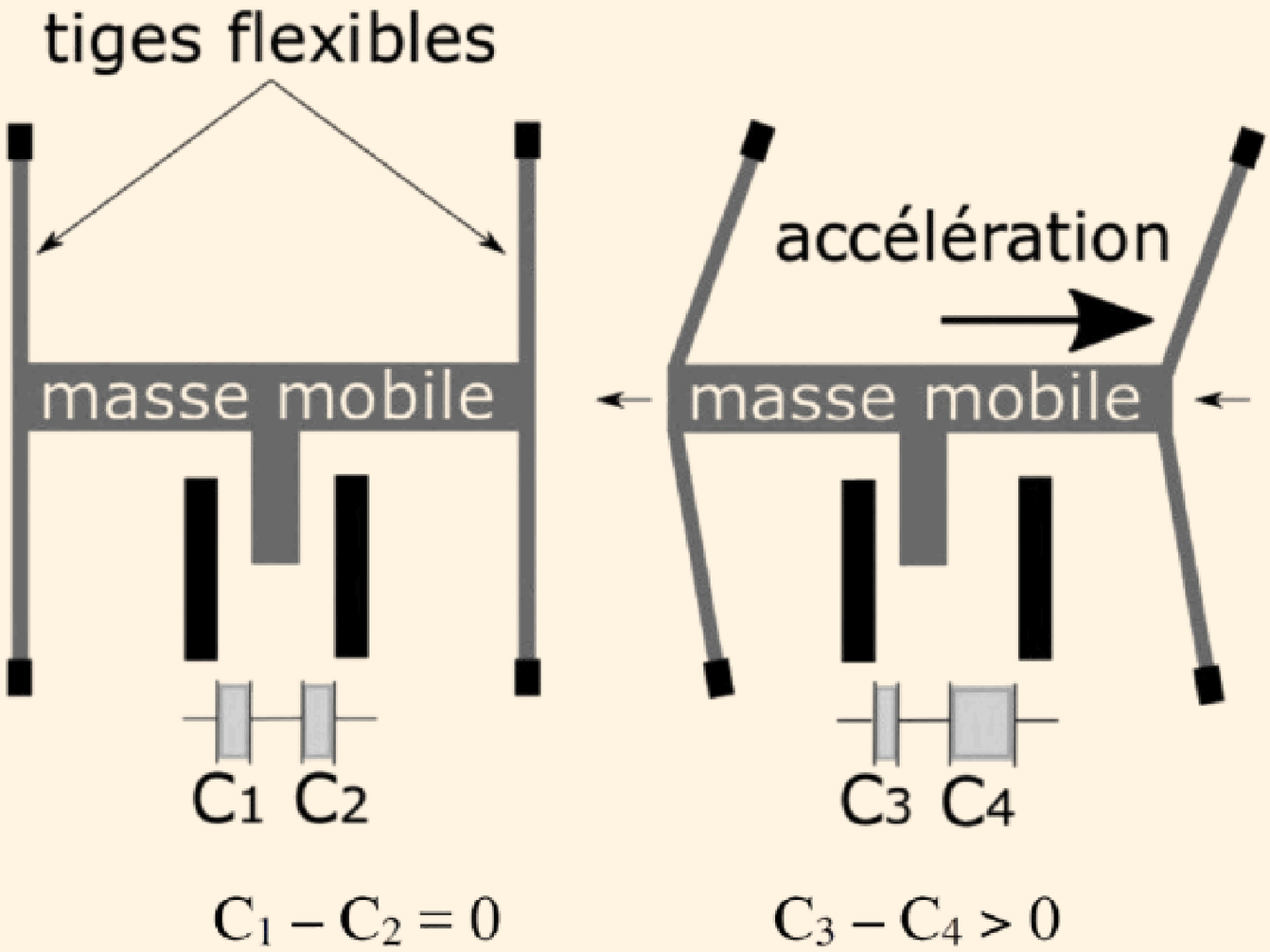

AL
KHAOUE
991

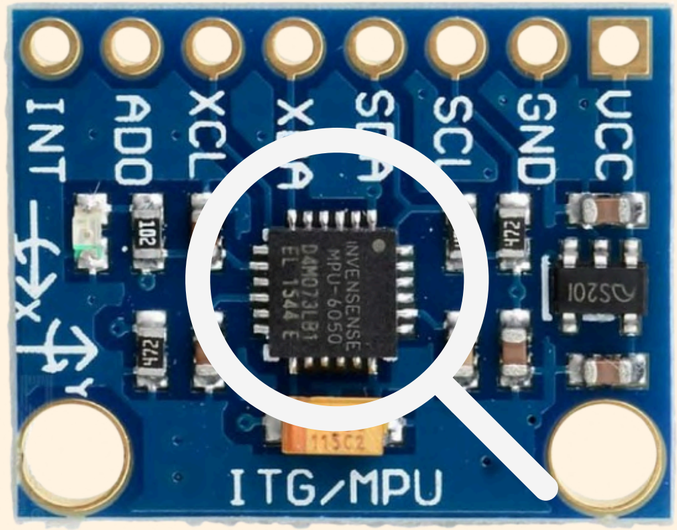
# Boucle d’asservissement

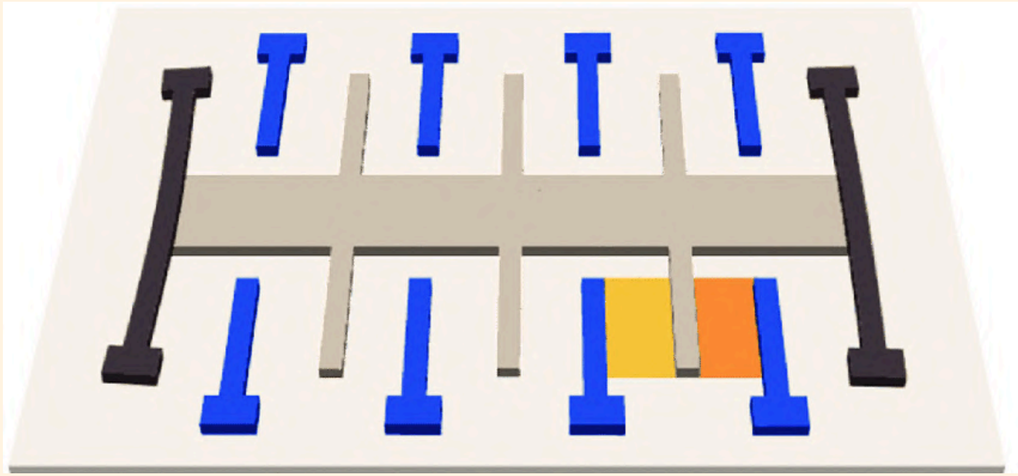



- $\alpha_c$ : Angle de consigne
- ε : Erreur
- Cmd : Commande
- α : Angle réel

# Comprendre le fonctionnement des accéléromètres

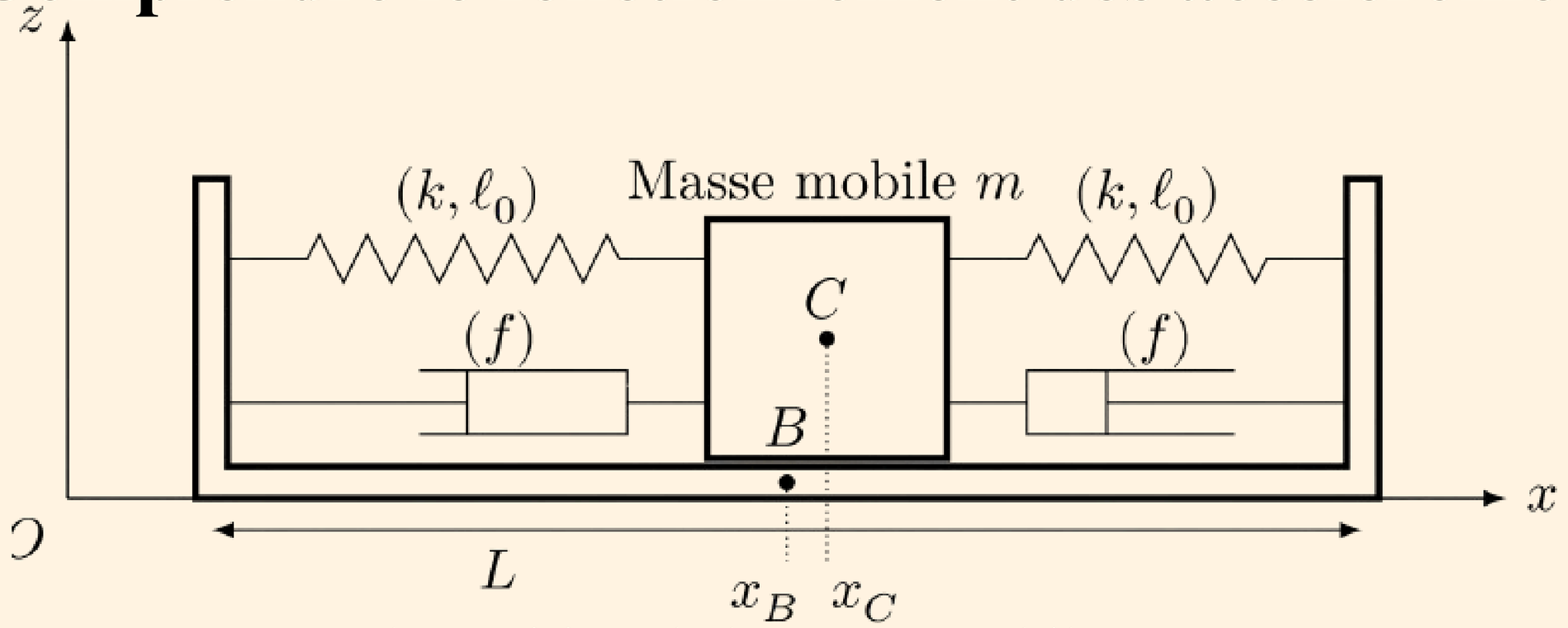


Source : editions-ellipses.fr

# Comprendre le fonctionnement des accéléromètres



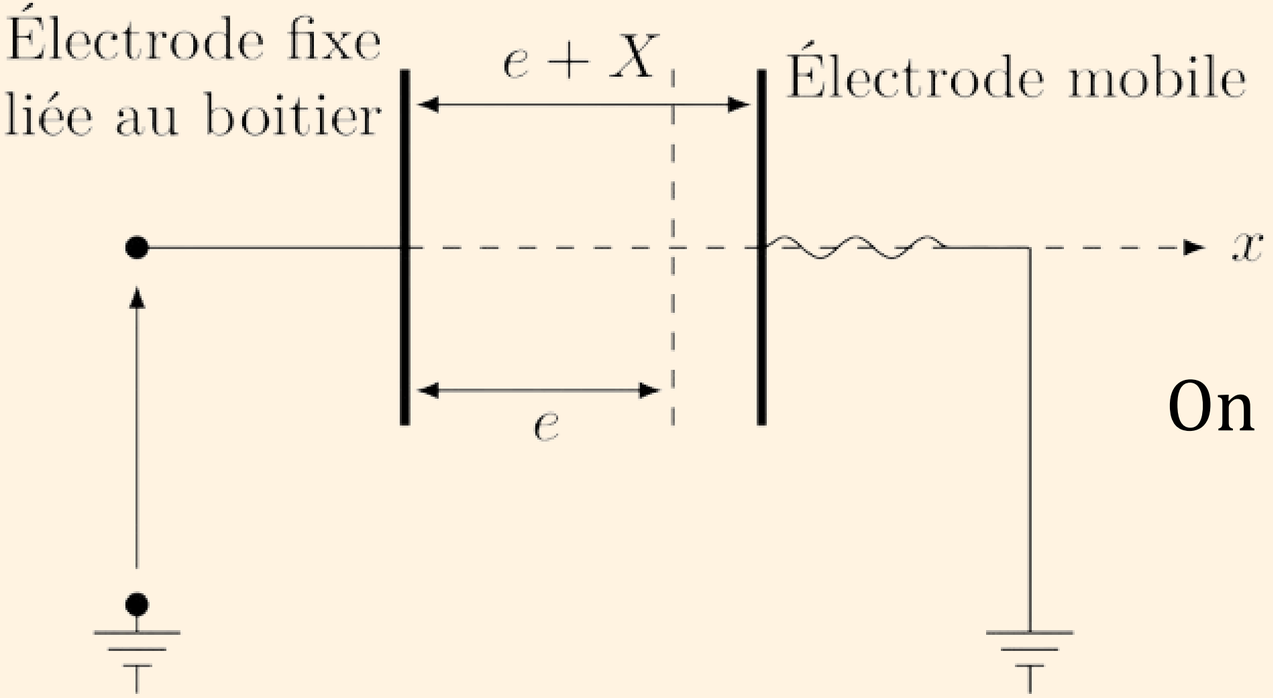

On note $X = x_c - x_b$, la position de la masse mobile par rapport au centre du boitier

---

$$\ddot{X} + \frac{\omega_0}{Q}\dot{X} + \omega_0^2 X = -a \Longrightarrow X(t) = -\frac{m}{2k} \cdot a(t)$$

Sous réserve d'être dans un certain domaine de fréquence

# Comprendre le fonctionnement des accéléromètres




$$C = C_0 \cdot \frac{e}{e+X}$$

On déduit des relations précédente que :

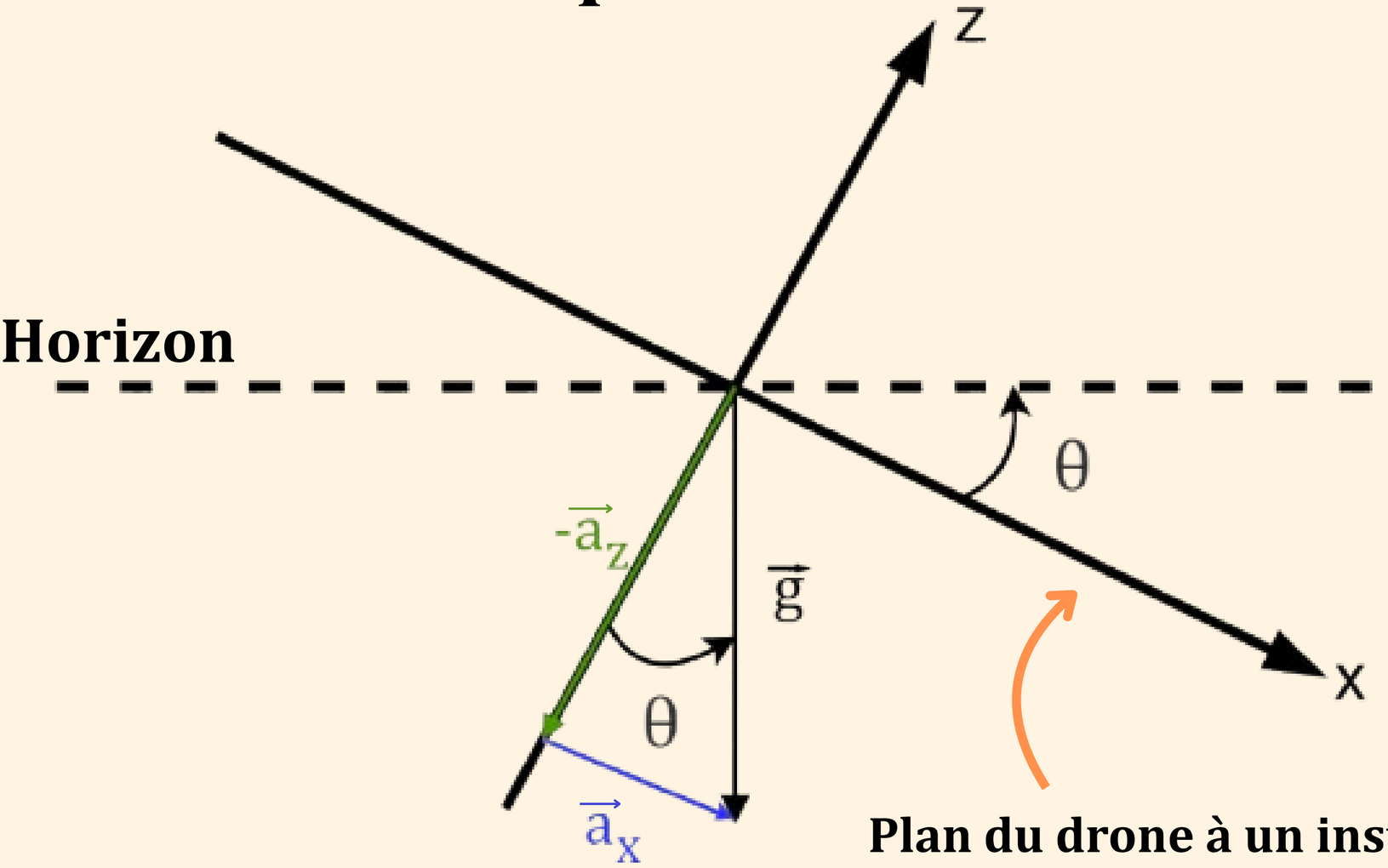
$$a = -2ke \cdot \frac{C_0 - C}{m}$$

- C : Capacité
- $C_0$ : Capacité à X = 0m
- k : Raideur du ressort
- e : Distance entre 2 électrode

**On retrouve donc l'accélération linéraire sur un axe**

# Pourquoi avoir besoin de l'accélération ?

$\overrightarrow{g}$ : accélération de la pesanteur



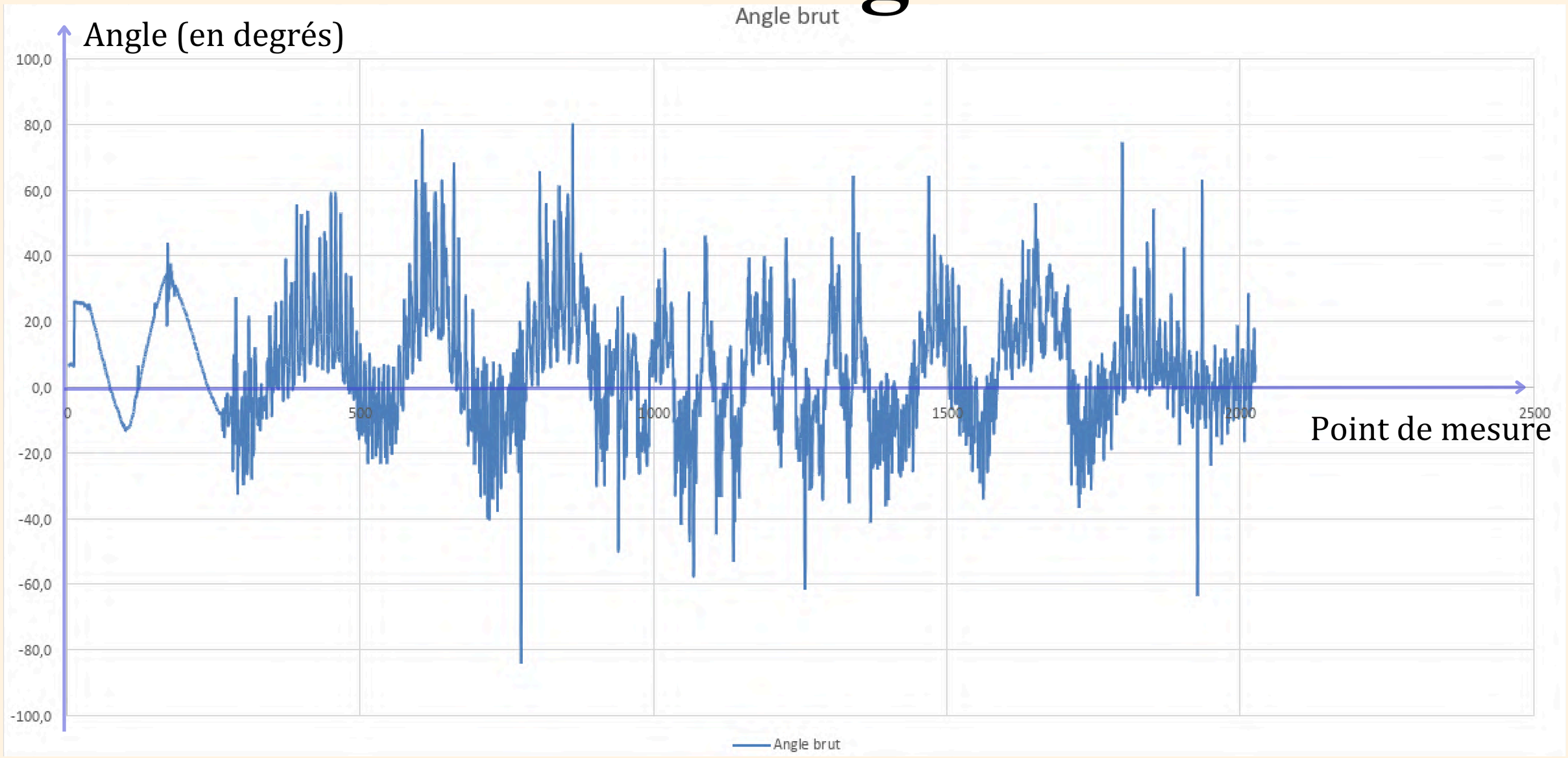

$$\overrightarrow{a_x} = g \cdot cos(\theta)\overrightarrow{u_x}$$

$$\overrightarrow{a_z} = -g \cdot sin(\theta)\overrightarrow{u_z}$$

$$\theta = \arctan(-\frac{a_x}{a_z})$$

**Plan du drone à un instant t**

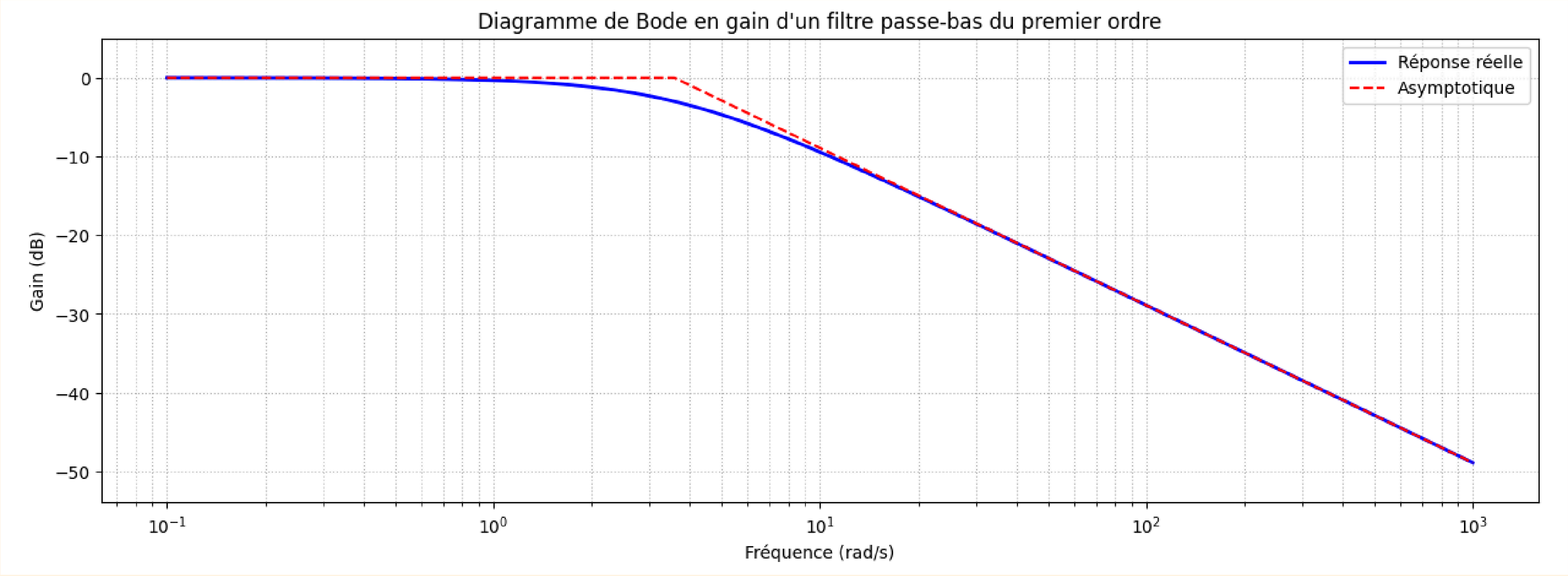
# Filtrage


Angle brut
Angle (en degrés)
100,0
80,0
60,0
40,0
20,0
0,0
-20,0
-40,0
-60,0
-80,0
-100,0
0
500
1000
1500
2000
2500
Point de mesure
Angle brut

# Filtre passe bas du premier ordre

$$\frac{\alpha_f}{\alpha_b} = \frac{1}{1+j\frac{\omega}{\omega_0}}$$

- $\alpha_f$ : Angle filtré
- $\alpha_b$ : Angle brut



# Fréquence des vibrations moteurs

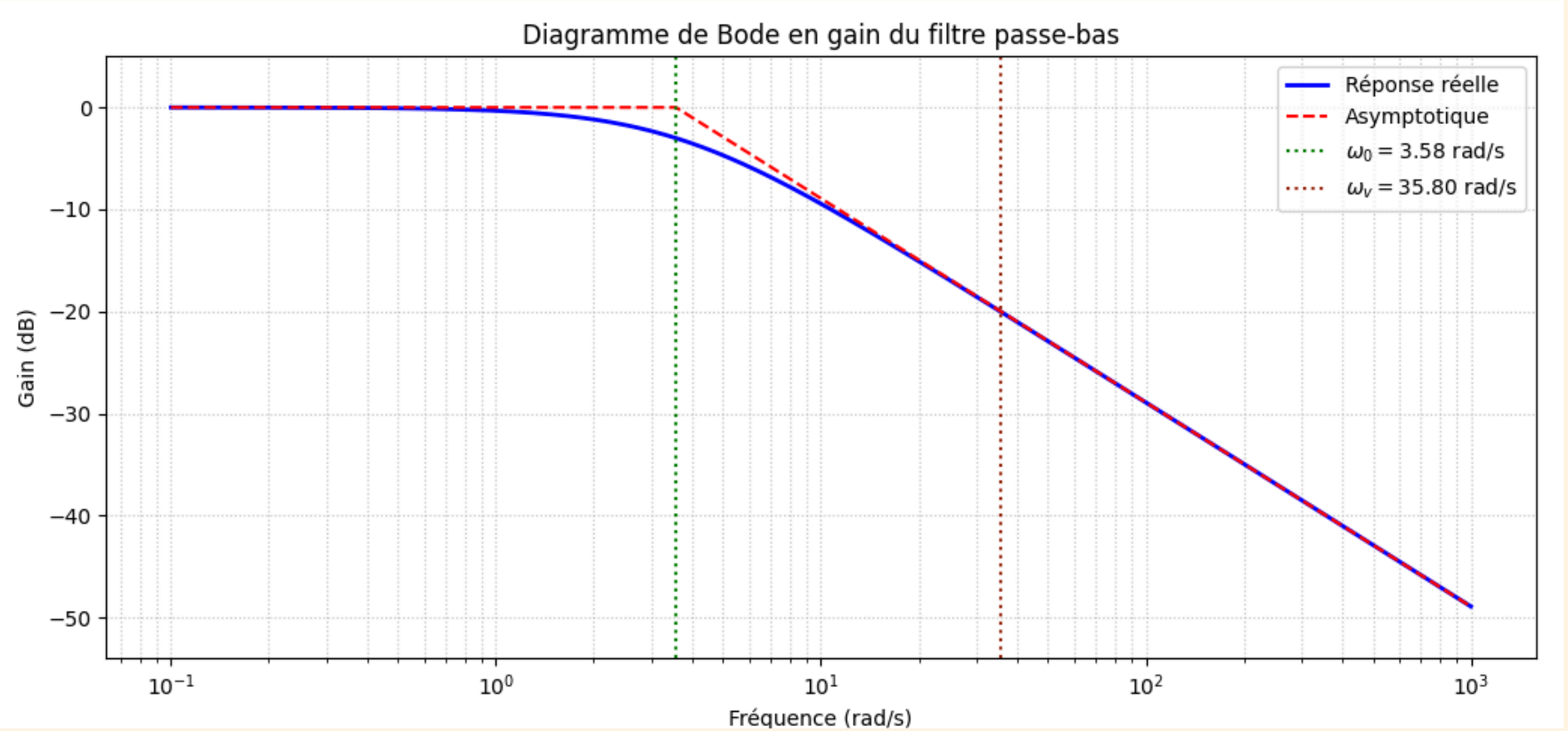



**On repère un motif qui se répète**

# Valeur de $\tau$

On obtient une fréquence de vibration $f_v$ = **<u>5,7 Hz</u>** soit $\omega_v$ = **<u>35 .81 rad/s</u>**

Dans le domaine de Laplace : $\frac{\alpha_f(p)}{\alpha_b(p)} = \frac{1}{1+\tau p}$ | Par identification : $\tau = \frac{1}{\omega_0} = \frac{1}{2\pi \cdot f_0}$



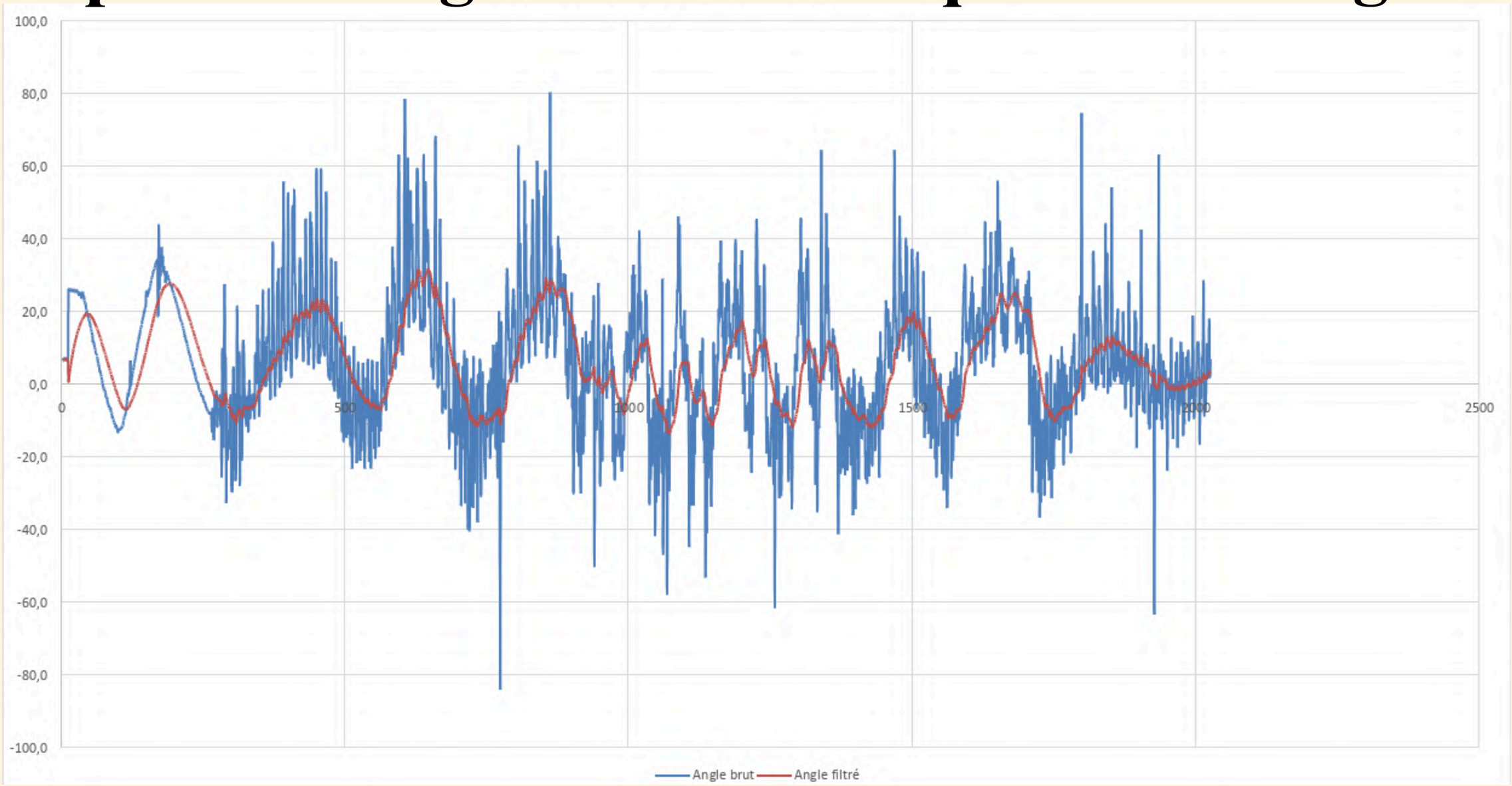

Une décade avant :

$$\omega_0 = \frac{\omega_v}{10}$$

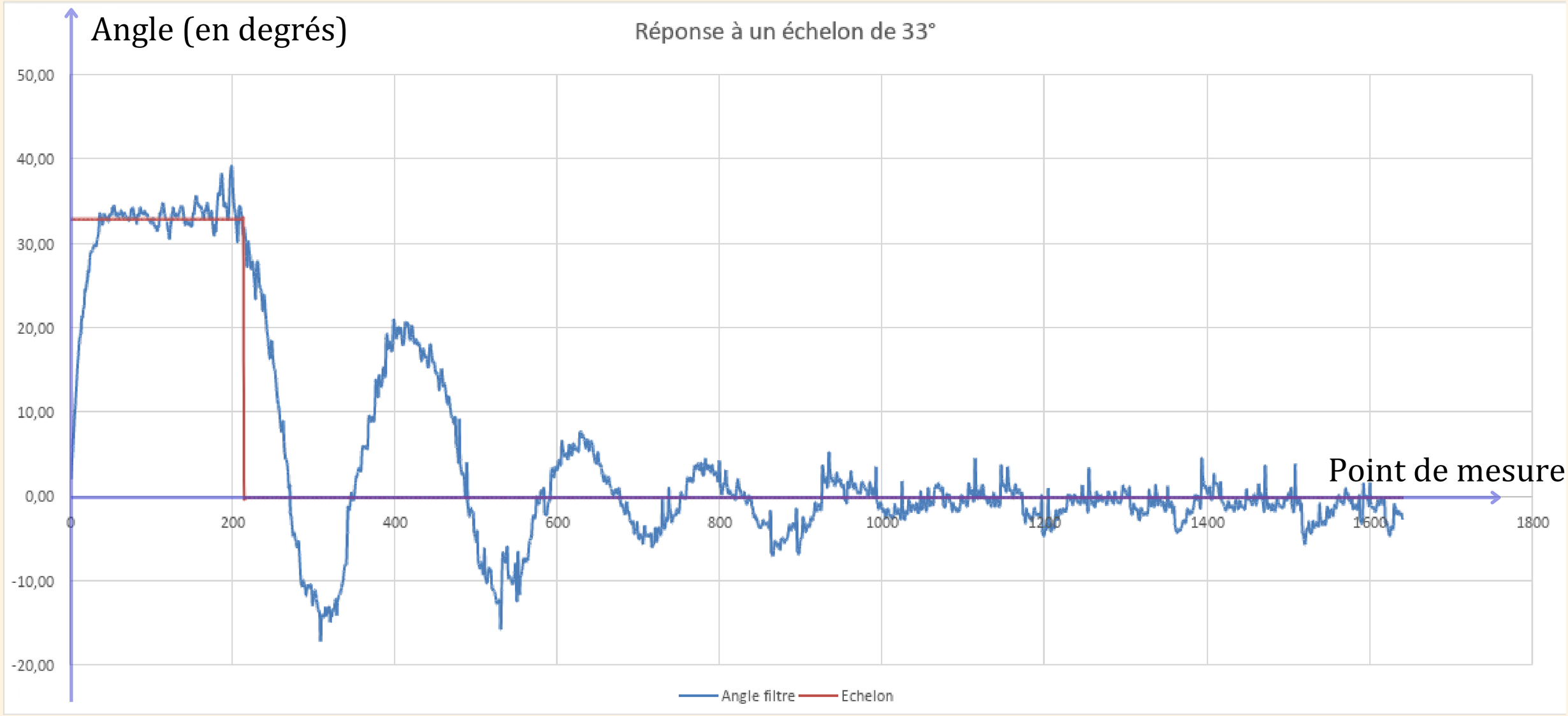
On obtient finalement :

$$\tau = 280\ ms$$

# Comparaison angle filtré numériquement et angle brut



# Réponse à un échelon angulaire



# Assimilation à un système du 2ème ordre

$$H(p) = \frac{K}{1 + \frac{2m}{\omega_0} \cdot p + \frac{p^2}{\omega_0^2}}$$

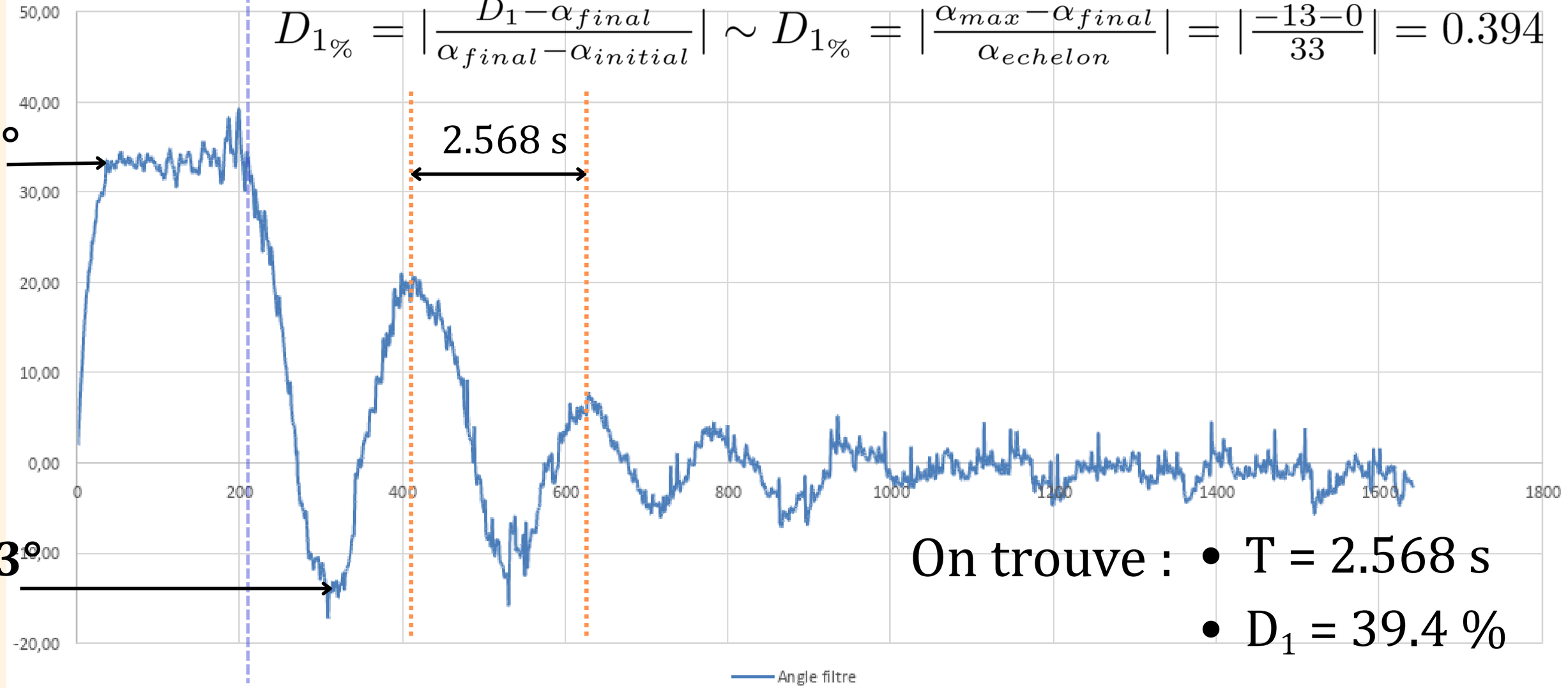
$$D_1 = e^{-\frac{\pi m}{\sqrt{1-m^2}}} \Leftrightarrow \boxed{m = \frac{\pm ln(D_1)}{\sqrt{\pi^2 + ln^2(D1)}}}$$

$$T = \frac{2\pi}{\omega_0 \cdot \sqrt{1-m^2}} \Leftrightarrow \boxed{\omega_0 = \frac{2\pi}{T \cdot \sqrt{1-m^2}}}$$

- m : Coefficient d'amortissement
- $\omega_0$ : Pulsation propre
- $D_1$ : Premier dépassement (relatif)
- T : Période des pseudos oscillations
- K : Gain statique

# $D_1$ et T

$$D_{1\%} = \left|\frac{D_1 - \alpha_{final}}{\alpha_{final} - \alpha_{initial}}\right| \sim D_{1\%} = \left|\frac{\alpha_{max} - \alpha_{final}}{\alpha_{echelon}}\right| = \left|\frac{-13-0}{33}\right| = 0.394$$




On trouve :
- T = 2.568 s
- $D_1$ = 39.4 %

# Assimilation à un système du 2ème ordre

$$m = \frac{\pm ln(D_1)}{\sqrt{\pi^2 + ln^2(D1)}} = \frac{-ln(0.394)}{\sqrt{\pi^2 + ln^2(0.394)}} = 0.284$$

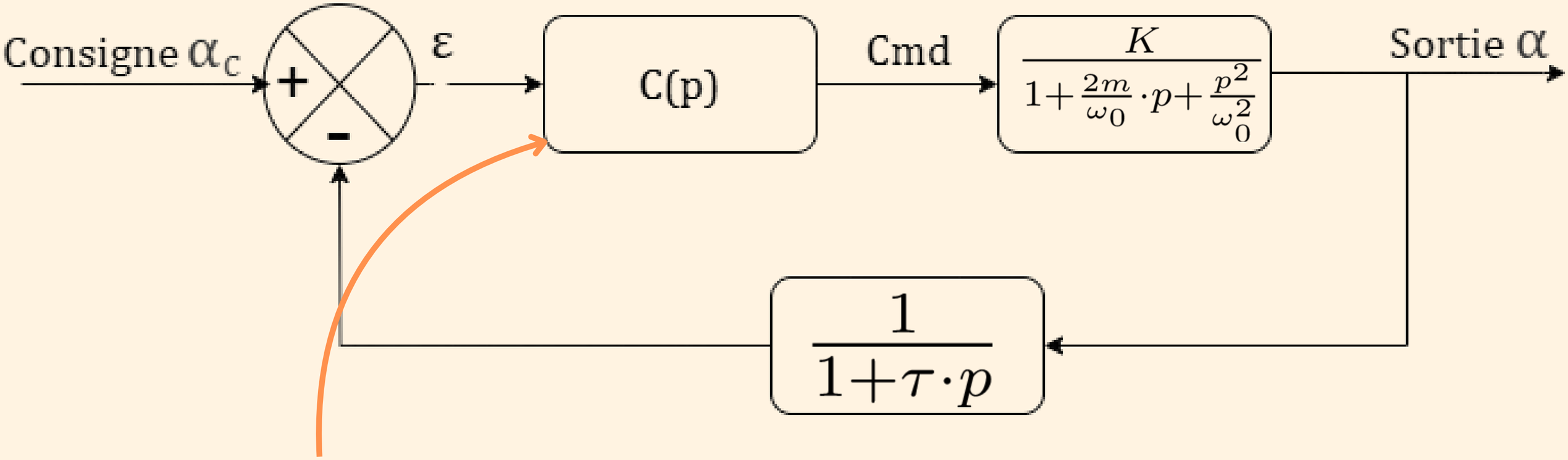
$$\omega_0 = \frac{2\pi}{T \cdot \sqrt{1-m^2}} = \frac{2\pi}{2{,}568 \cdot \sqrt{1-(0.284)^2}} = 2.551\ rad/s$$

$$K = 1$$

---

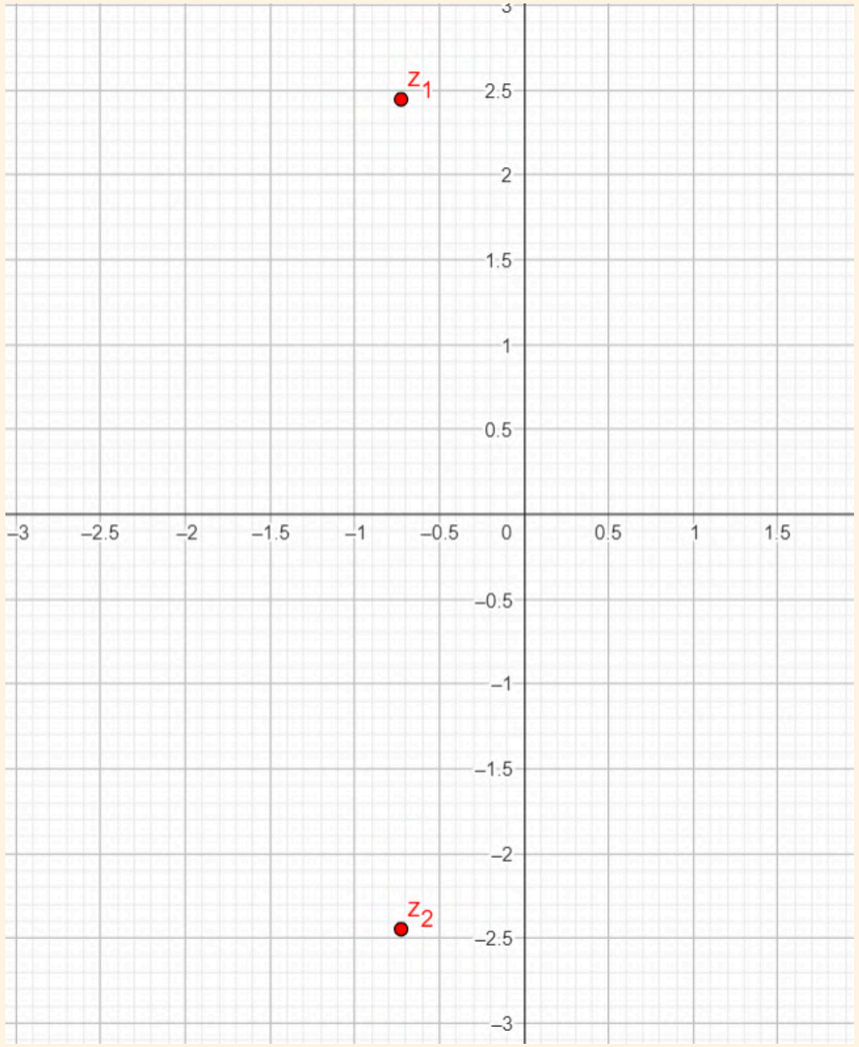
On a donc : $$H(p) = \frac{1}{1 + 0.22285 \cdot p + 0.15363 \cdot p^2}$$

# Boucle d’asserivsement




Intéressons nous maintenant au correcteur

# Pôle dominant ?




## Le correcteur PI :

$$C(p) = K \cdot \frac{1+\tau_i p}{\tau_i p}$$

- Augmente la précision, annule l'erreur statique
- Avec K >1 la rapidité est augmenté
- Dégrade la stabilité

# Fonction de transfert en boucle ouverte

$$F_{TBO}(p) = \underbrace{\frac{1}{1+0.22285p+0.15363p^2}}_{\text{Notre système}} \cdot \underbrace{\frac{1}{1+0.28p}}_{\text{Filtre passe-bas}}$$

Problème de stabilité → marge de gain et/ou marge de phase faible

---

$M_\varphi < 30° / M_G < 3dB$ : Système peu robuste

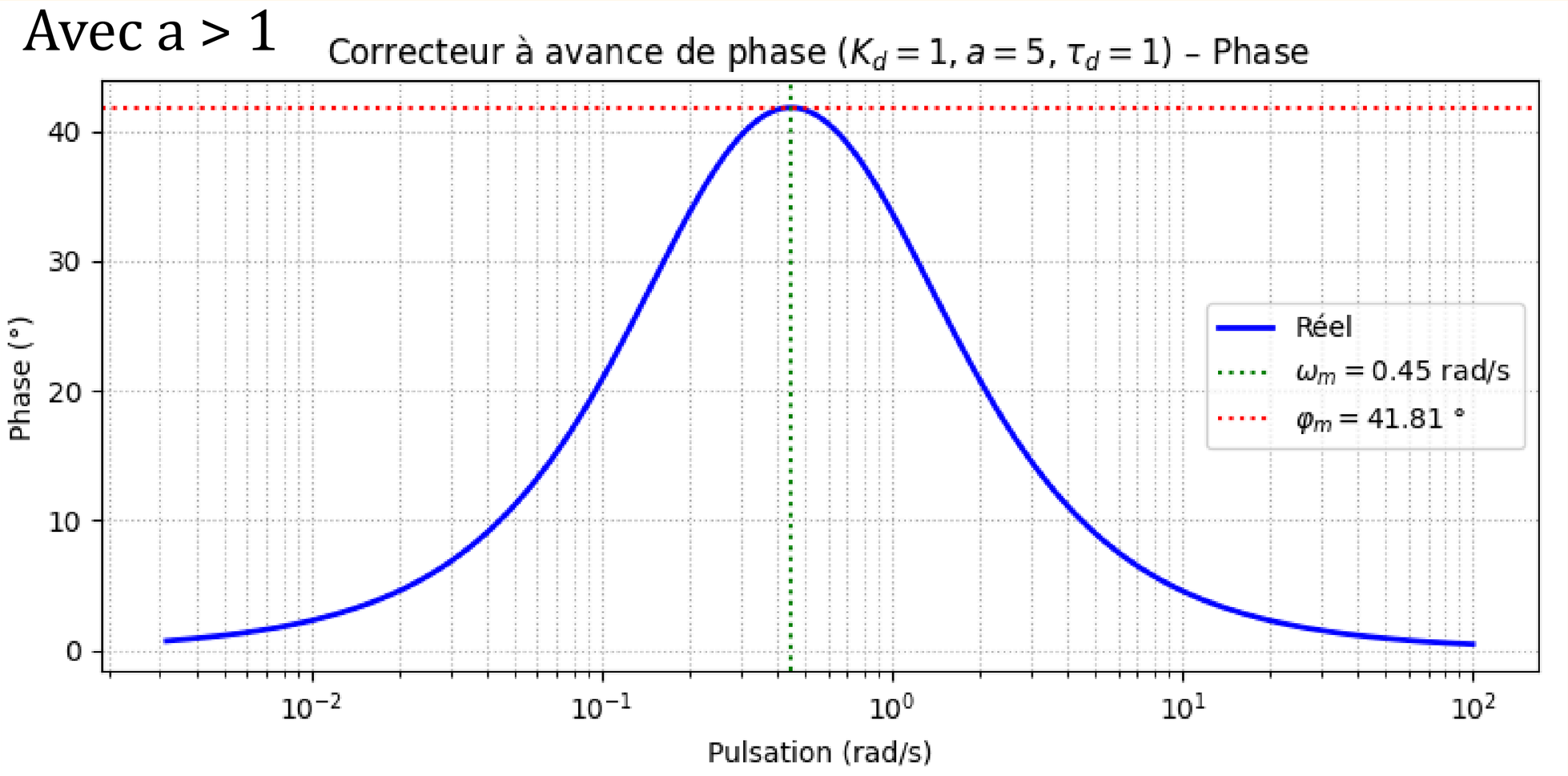
$M_\varphi > 60° / M_G > 12dB$ : Système stable mais réponse lente

**$M_\varphi = 45° / M_G = 10$ dB** est un bon compromis

# Le correcteur à avance de phase

$$C(p) = K_d \cdot \frac{1+a\cdot\tau_d p}{1+\tau_d p} \quad \Big| \quad \varphi_m = arcsin(\frac{a-1}{a+1}) \Leftrightarrow \boxed{a = \frac{1+sin(\varphi_m)}{1-sin(\varphi_m)}}$$

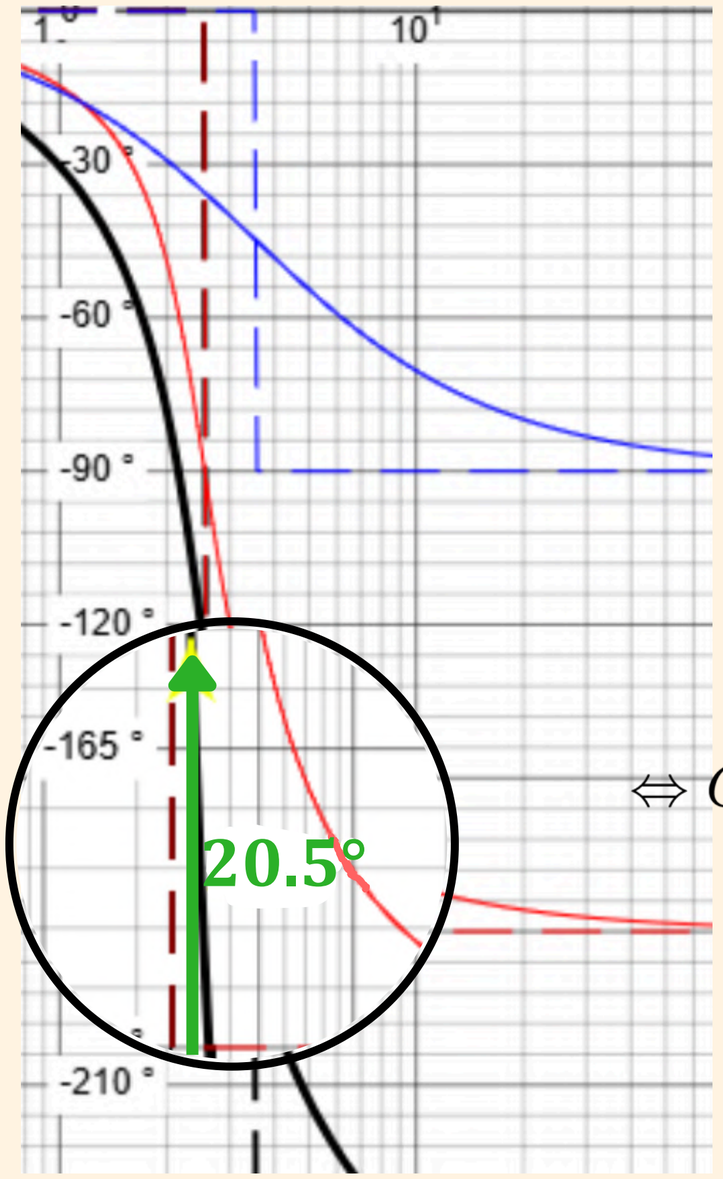
Avec a > 1




$$\omega_m = \frac{1}{\tau_d \cdot \sqrt{a}}$$

$$\Leftrightarrow$$

$$\boxed{\tau_d = \frac{1}{\omega_m \cdot \sqrt{a}}}$$




Courbe tracée à l'aide de bode.allais.eu

$$\boxed{\varphi_m = M_{\varphi_{voulue}} - M_{\varphi_{actuelle}}} \Leftrightarrow \varphi_m = 45 - 20.5 = 24.5^\circ$$

D'où : $$a = \frac{1+sin(\varphi_m)}{1-sin(\varphi_m)} = \frac{1+sin(24.5)}{1-sin(24.5)} = \boxed{2.417}$$

$$\boxed{\omega_m = \omega_{0dB} = 2.991\ rad/s}$$

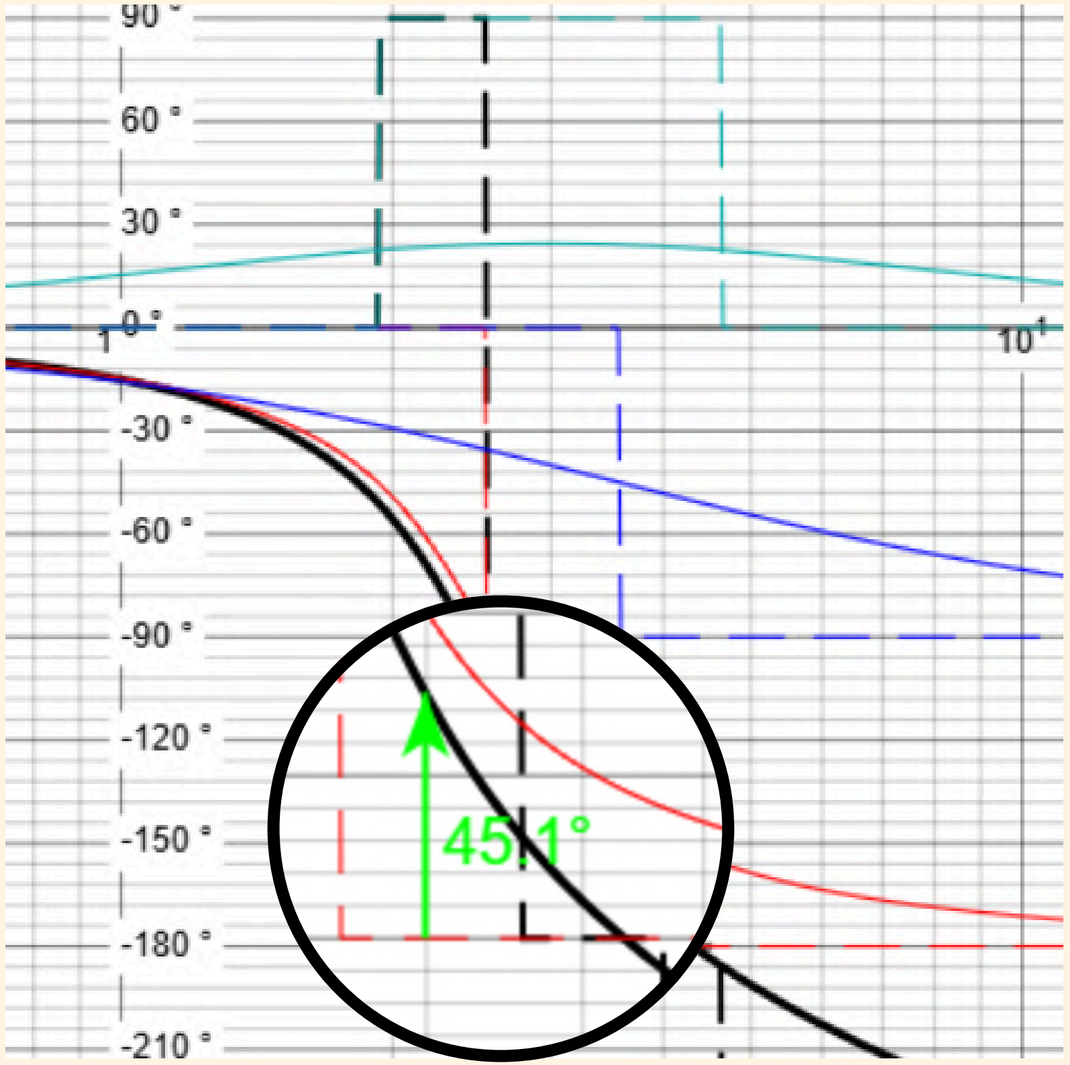
Alors : $$\tau_d = \frac{1}{\omega_m \cdot \sqrt{a}} = \frac{1}{2.991 \cdot \sqrt{2.417}} = \boxed{0.215\ s}$$

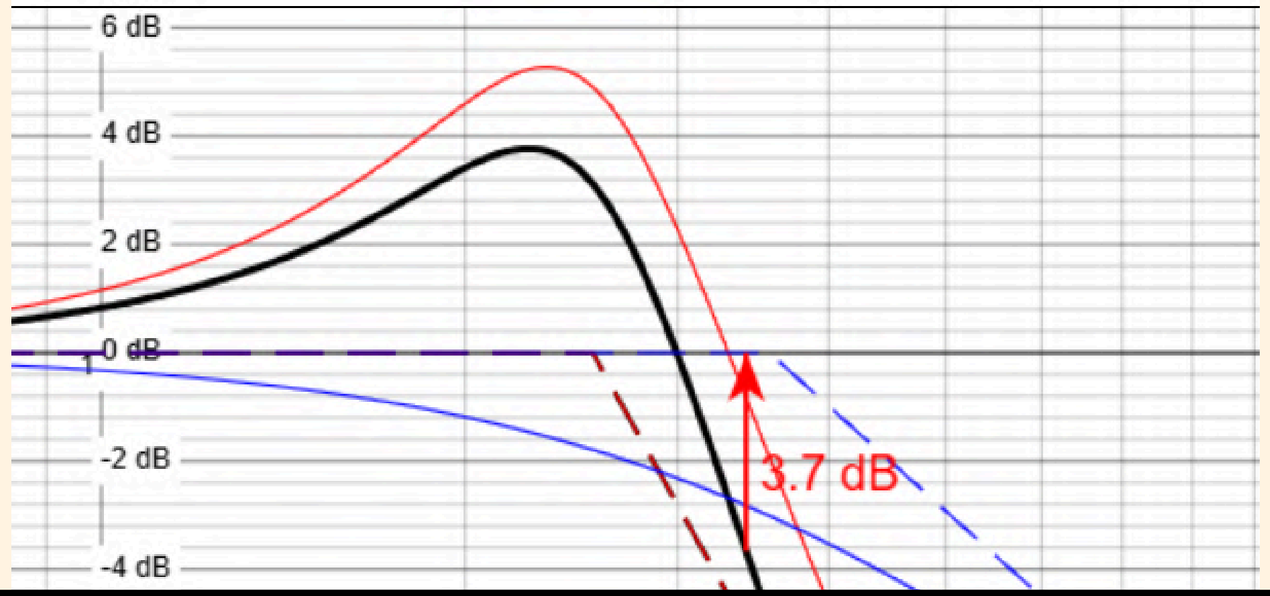
On cherche à garder : $\boxed{G_{dB}(j\omega_{0dB}) = 20\ log(|F_{TBO}(j\omega_{0dB})|) = 0\ dB}$

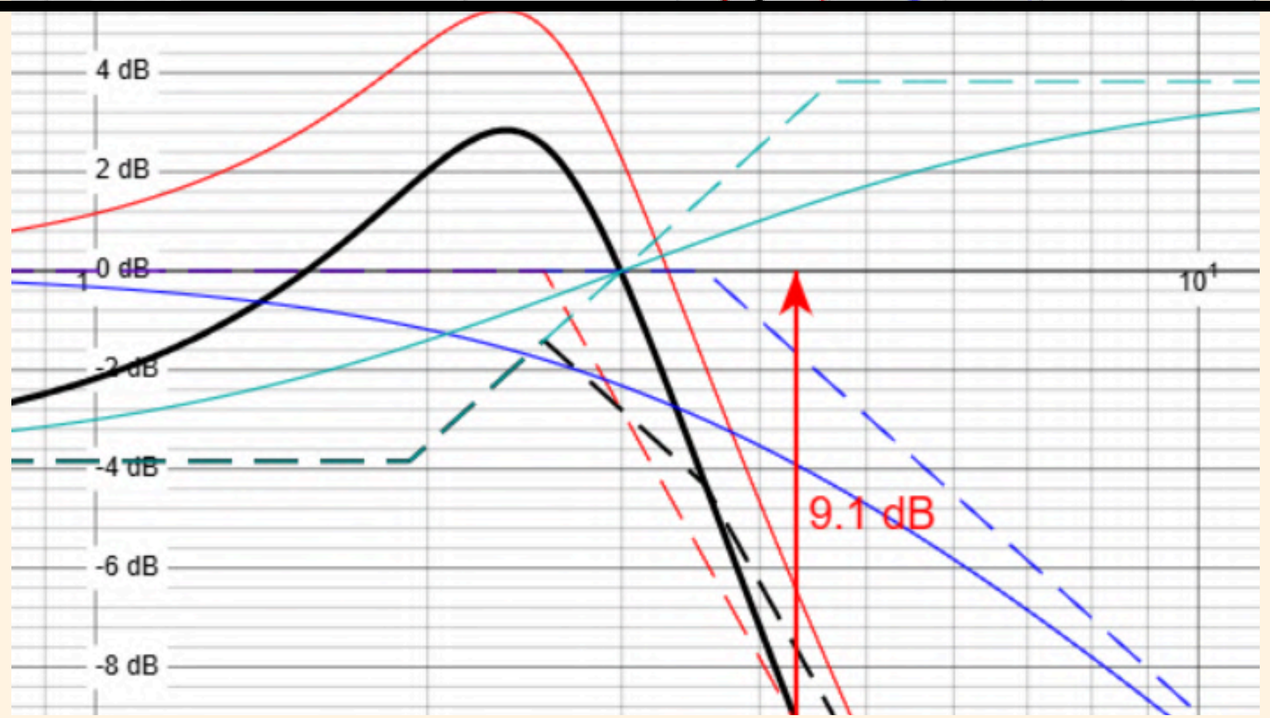
$$\Leftrightarrow G_{dB}(j\omega_{0dB}) = 20\ log(|K_d \frac{1+a\tau_d \cdot j\omega_{0dB}}{1+\tau_d j\omega_{0dB}} \frac{1}{1+\frac{2m}{\omega_0} j\omega_{0dB} + \frac{(j\omega_{0dB})^2}{\omega_0^2}} \frac{1}{1+\tau j\omega_{0dB}}|) = 0$$

$$\Leftrightarrow \quad K_d = \frac{|(1+\tau_d j\omega_{0dB}) \cdot (1+\frac{2m}{\omega_0} j\omega_{0dB} + \frac{(j\omega_{0dB})^2}{\omega_0^2}) \cdot (1+\tau j\omega_{0dB})|}{|1+a\tau_d \cdot j\omega_{0dB}|}$$

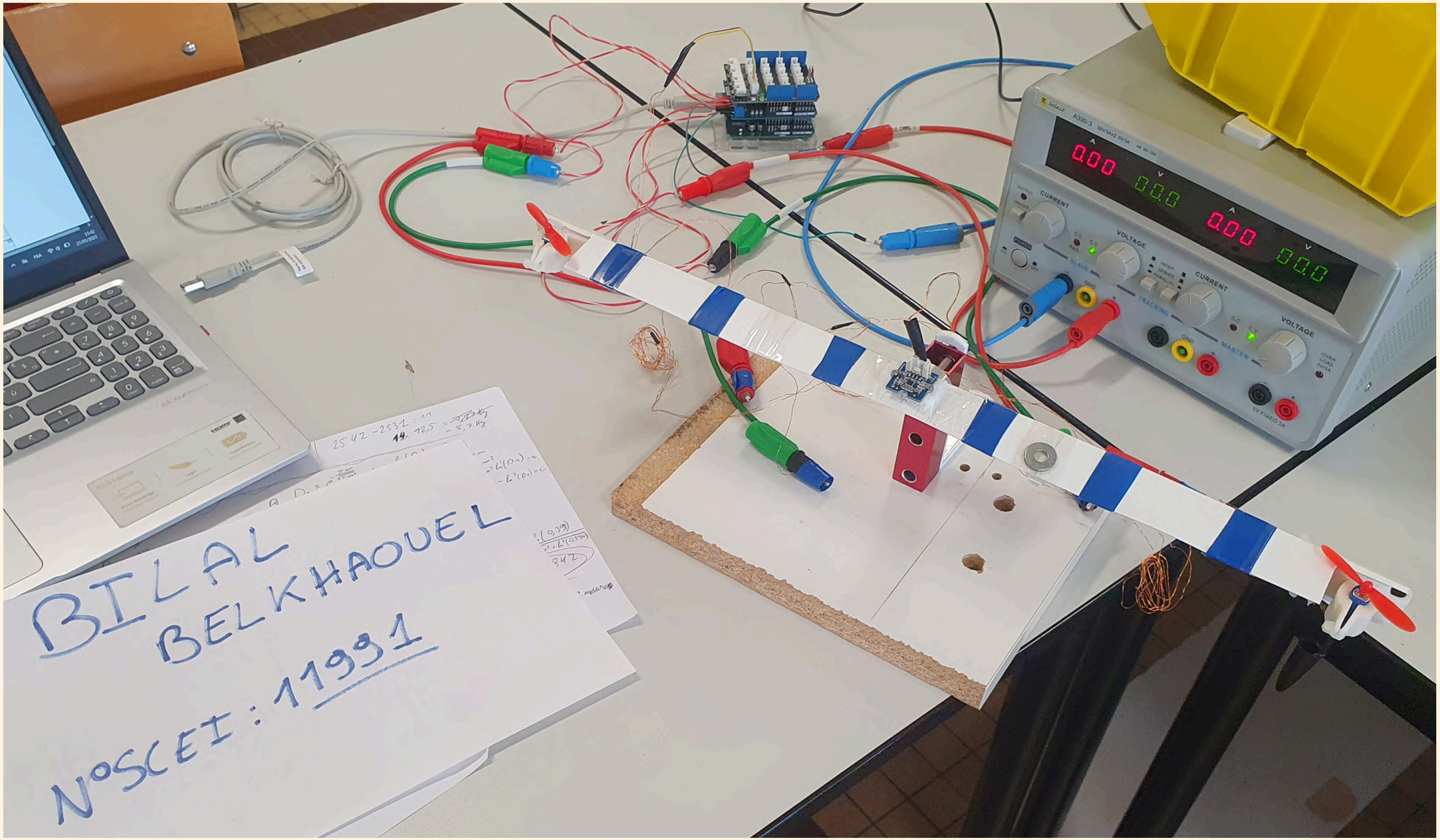
$$\boxed{K_d = 0.6424}$$



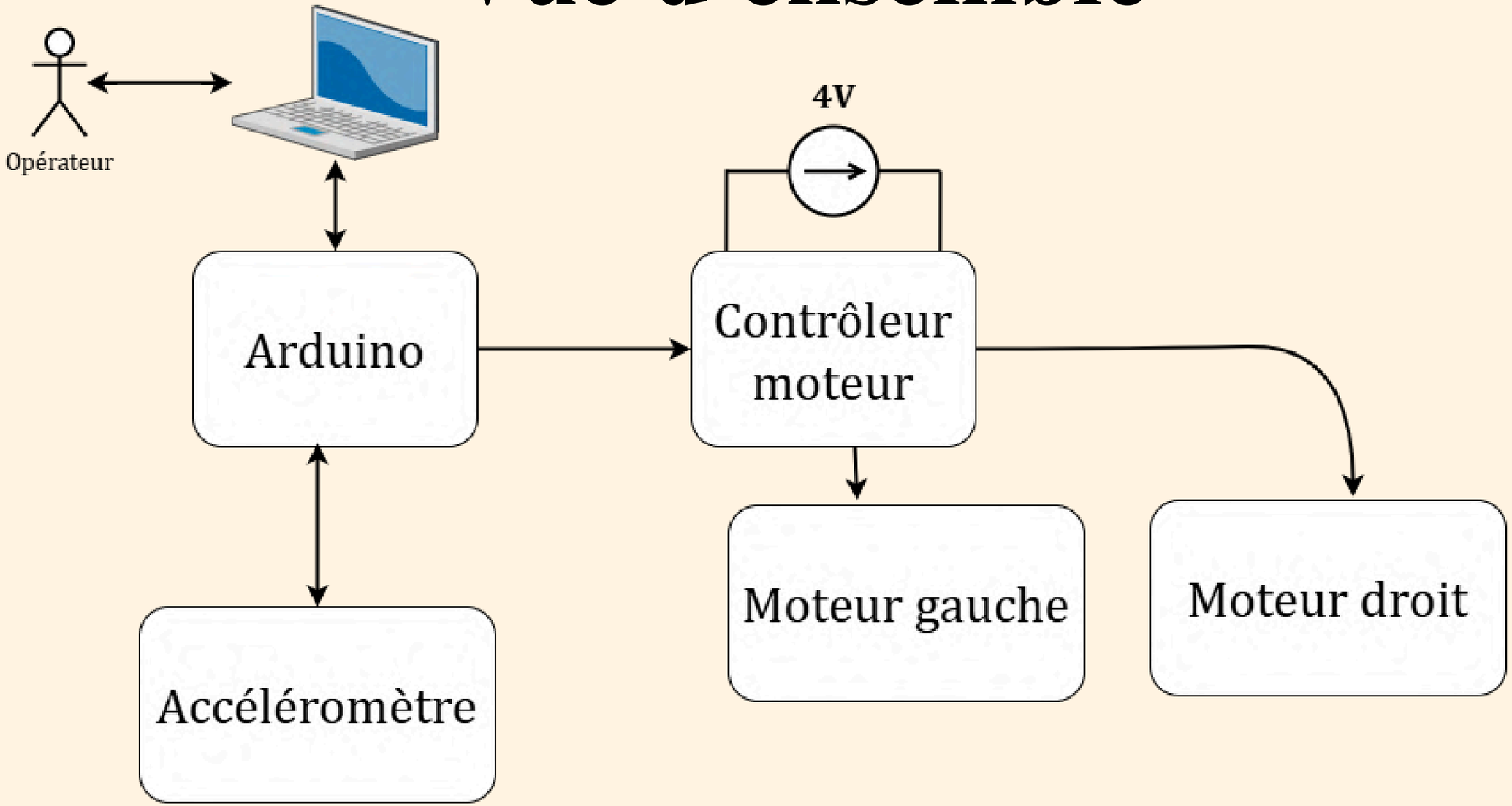




Courbe tracée à l'aide de bode.allais.eu


BILAL
BELKHAOUEL
N°SCEI : 11991
CURRENT
VOLTAGE
CURRENT
VOLTAGE
TRACKING
MASTER
SLAVE

# Vue d’ensemble



# Schéma d'Euler explicite

Dans le domaine continu : $\frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$

Dans le domaine discret : $f'[i] = \frac{f[i]-f[i-1]}{Te}$

---

$$\frac{s(p)}{e(p)} = \frac{1}{1+\tau p} \Rightarrow \tau \frac{ds(t)}{dt} + s(t) = e(t)$$

$$\tau \frac{s[i]-s[i-1]}{Te} + s[i] = e[i] \Leftrightarrow \boxed{s[i] = \frac{e[i]+\frac{\tau}{Te} \cdot s[i-1]}{\frac{\tau}{Te}+1}}$$

$$\frac{cmd(p)}{\varepsilon(p)} = K_d \frac{1+a\tau_d p}{1+\tau_d p} \Rightarrow \boxed{cmd[i] = \frac{K \cdot ((T_e+a\tau_d)\varepsilon[i]-a\tau_d\varepsilon[i-1])+\tau_d \cdot cmd[i-1]}{T_e+\tau_d}}$$

# Mise en place dans Arduino IDE

```
17  float angleFiltre;
18  float angleFiltre_precedent = 0;
```

```
22  float erreur ;
23  float erreur_precedente = 0;
24  /* Par Bilal BELKHAOUEL CPGE TSI n°SCEI : 11991 */
25  float consigne = 0.0;                // Drone à plat
```

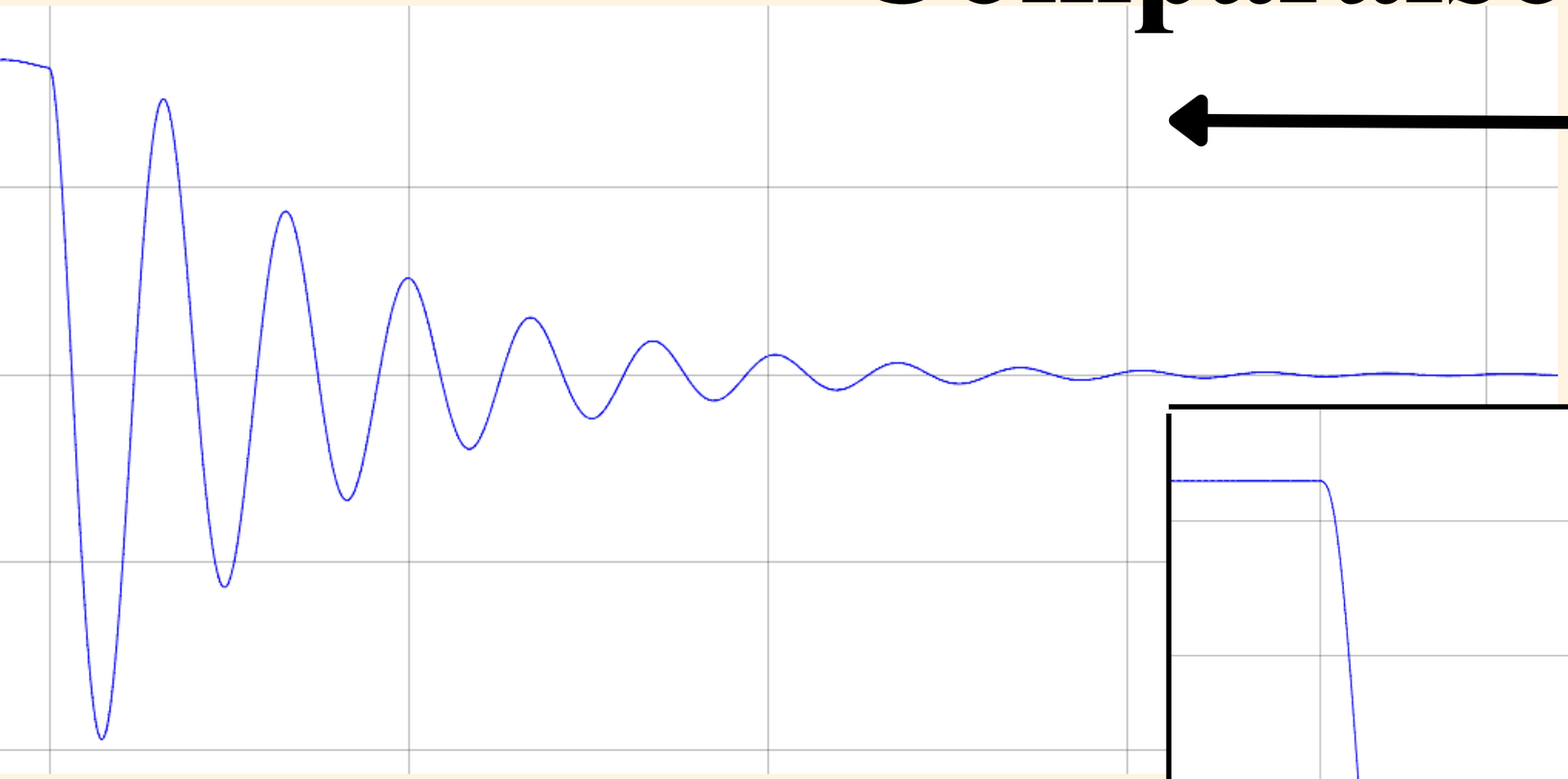
- deltaT = $T_e$
- angleBrut est calculé avec la formule vu précédemment

```
angleFiltre = (angleBrut + angleFiltre_precedent*(tau/deltaT))/((tau/deltaT) + 1);
angleFiltre_precedent = angleFiltre;
```

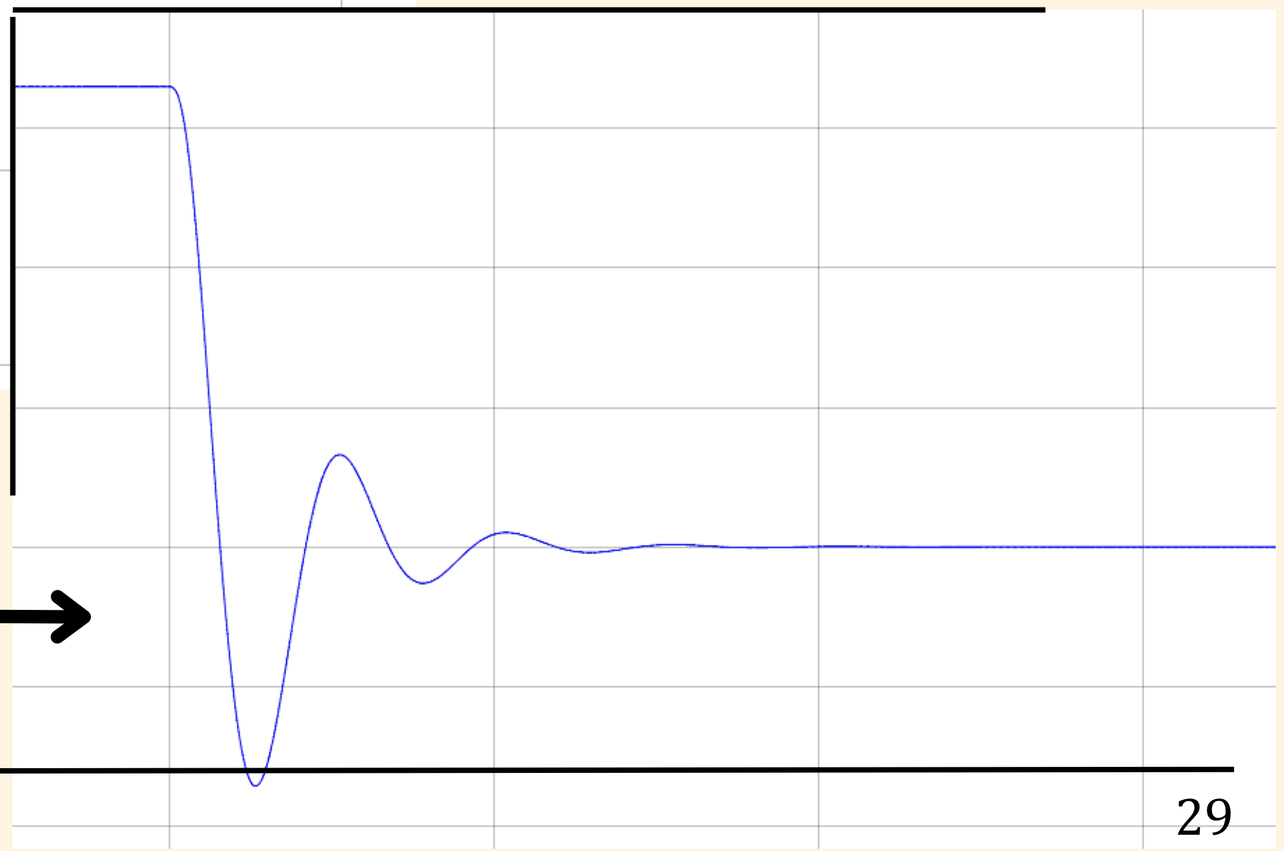
```
85  erreur = consigne - angleFiltre;
86  erreur_precedente = erreur;
```

```
88  commande = (K*((deltaT*0.001 + a*taud)*erreur - a*taud*erreur_precedente)
89  + taud*commande_precedente)/(deltaT*0.001 + taud);
90  commande_precedente = commande;
```

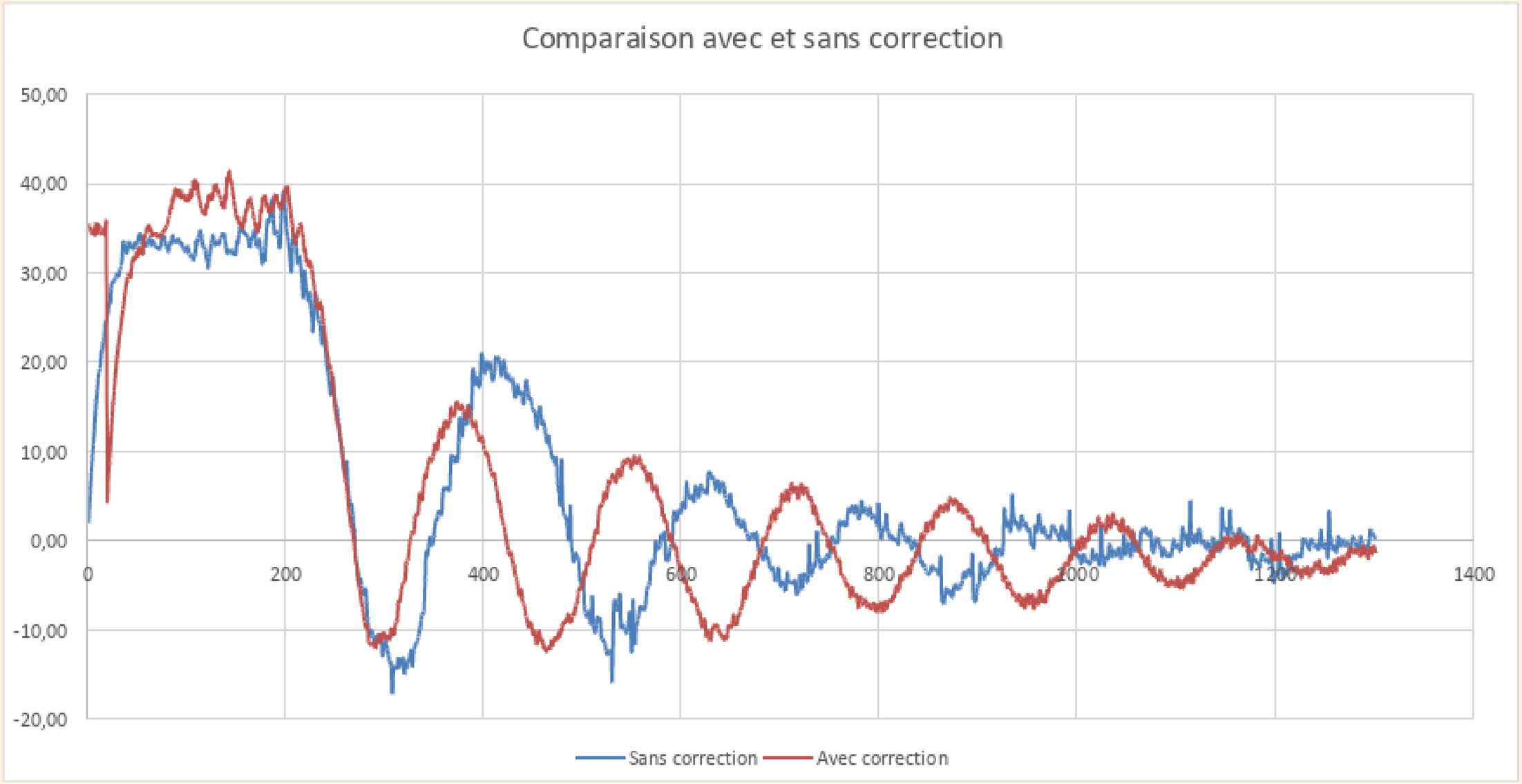
# Comparaison



Sans correction



Avec correction


Comparaison avec et sans correction
50,00
40,00
30,00
20,00
10,00
0,00
-10,00
-20,00
0
200
400
600
800
1000
1200
1400
Sans correction
Avec correction

# Conclusion et piste d'amélioration

- Système présentant des non-linéarités :
  - Butée mécanique
  - Fils qui tirent les moteurs
  - Déséquilibre mécanique → un coté plus lourd que l'autre
- Pas totalement invariant :
  - Le temps d'échantillonnage varie et grand (12.4 ms)
  - Premier dépassement varie(non linéarité) → m et $\omega_0$ change
- Les moteurs n'ont pas exactement la même force de poussé



# Merci pour votre écoute !

## Avez-vous des questions ?

# Annexe : Transmission, Transformation, Conversion

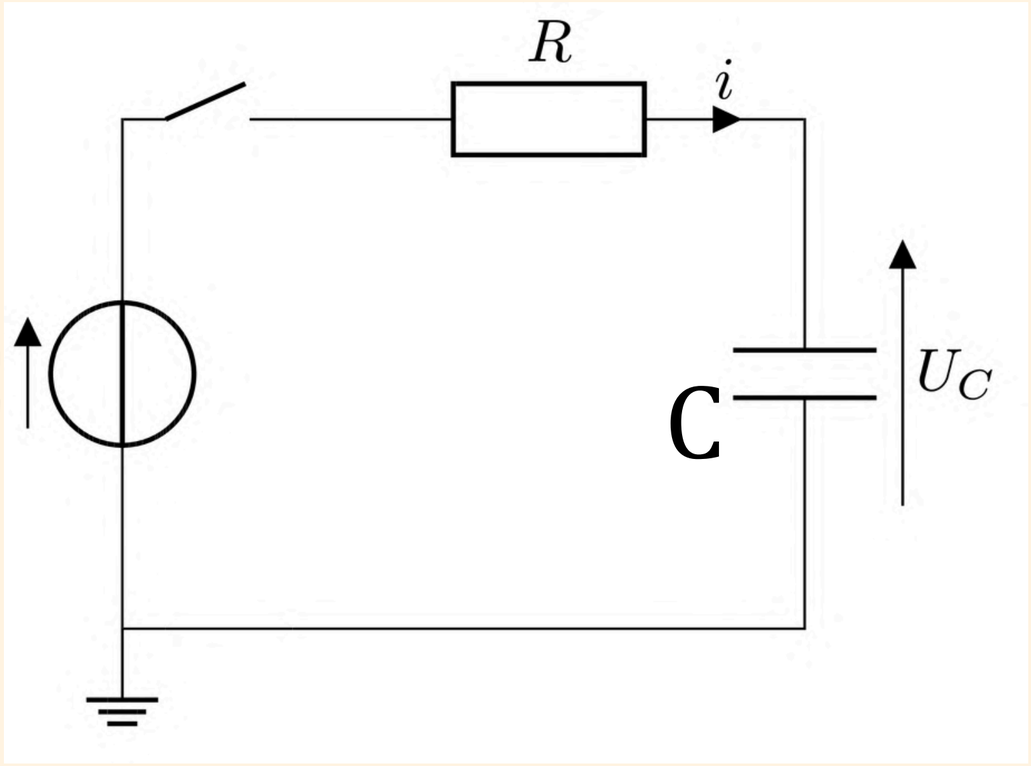
**Conversion :**

Accéléromètre → Instrument fondamentale d’un drone : énergie cinétique convertit en énergie électrique

**Transition:**

Méthode de transport, de sécurité, de prévention et de défense

# Annexe : Mesure de capacité



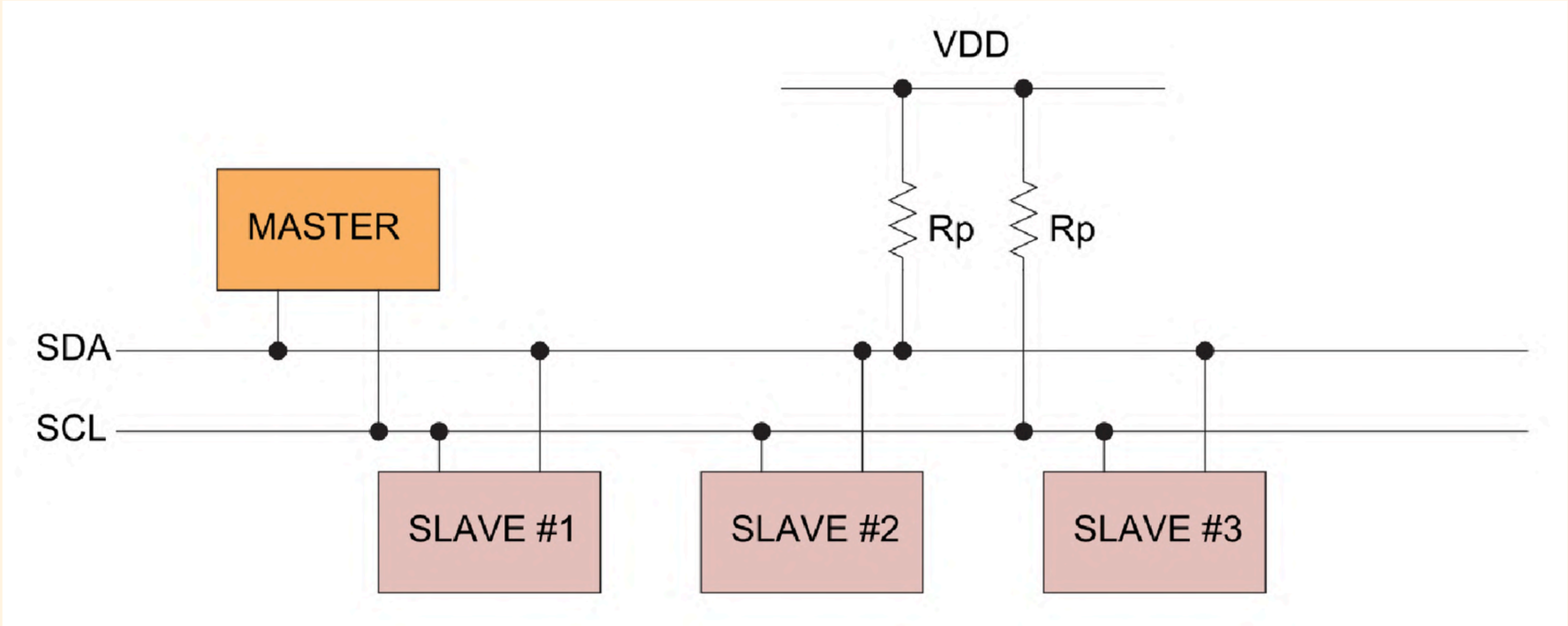

$$U_c(t) = E(1 - e^{-\frac{t}{\tau}})$$

Avec : $$\tau = \frac{1}{RC} \Leftrightarrow C = \frac{1}{R\tau}$$

De plus :

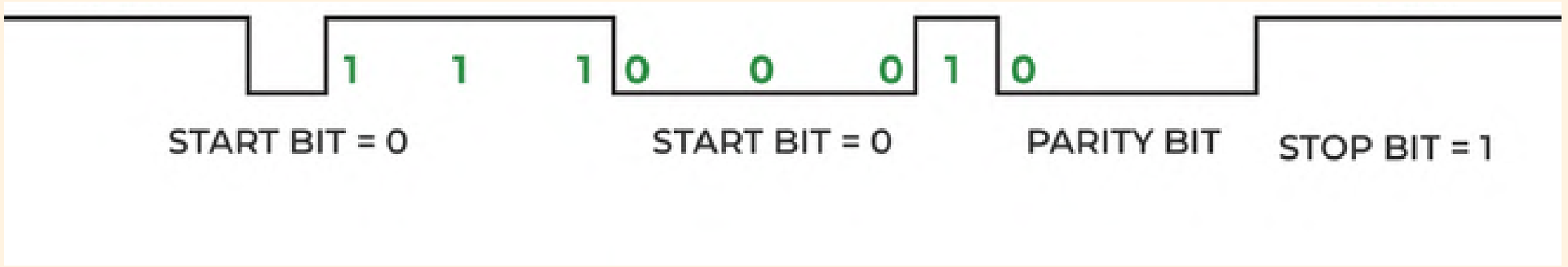
$$U_c(\tau) = 0.63 \cdot E$$

# Annexe : La transmission I²C



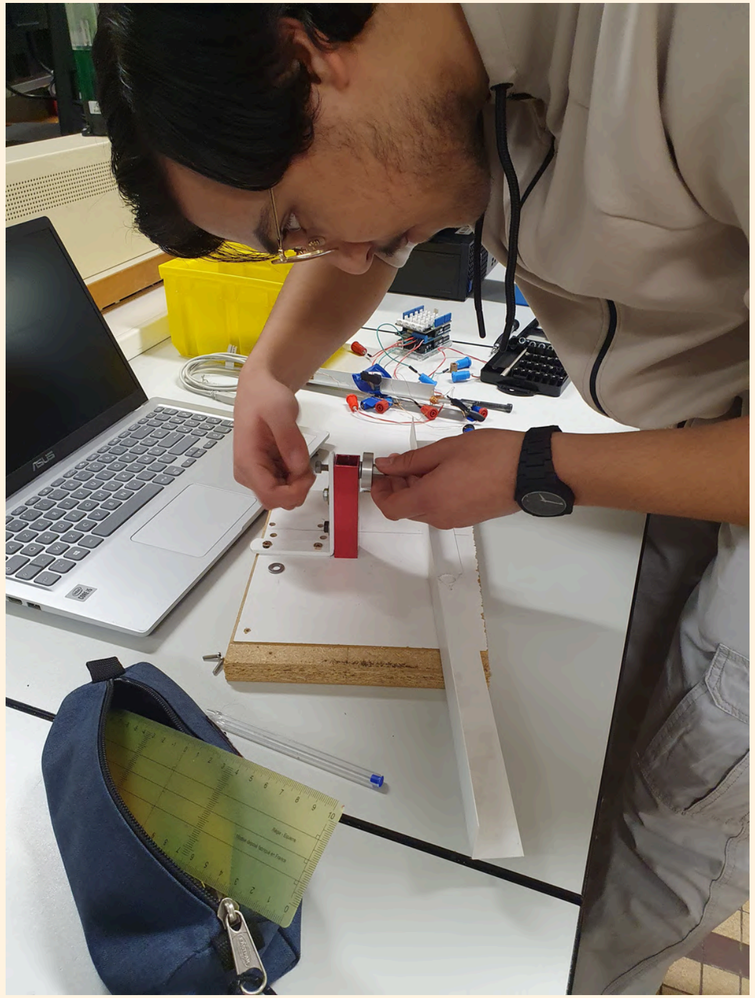

Source : Analog Device

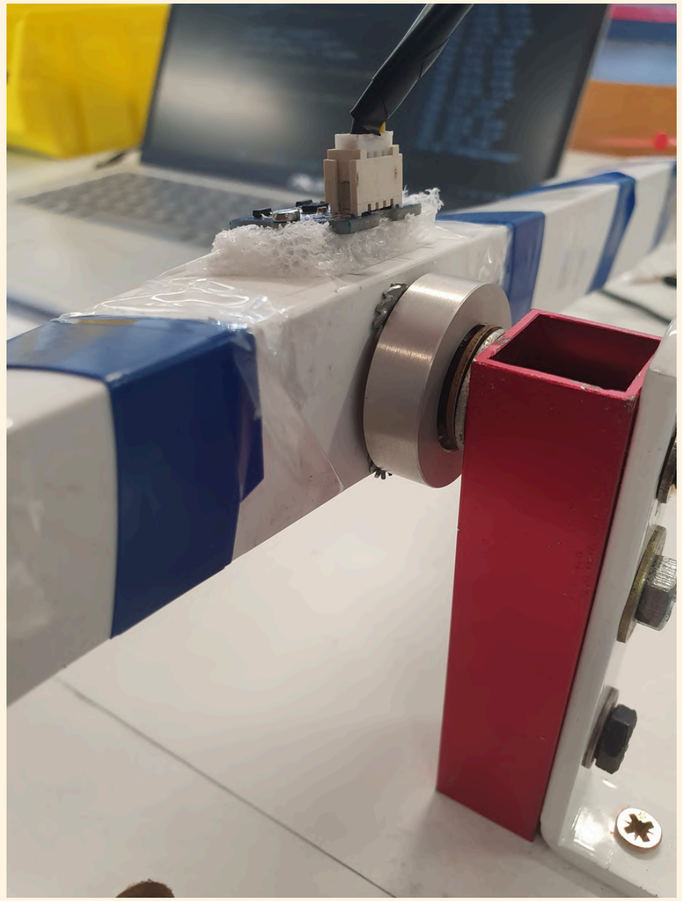
# Annexe : La Transmission UART

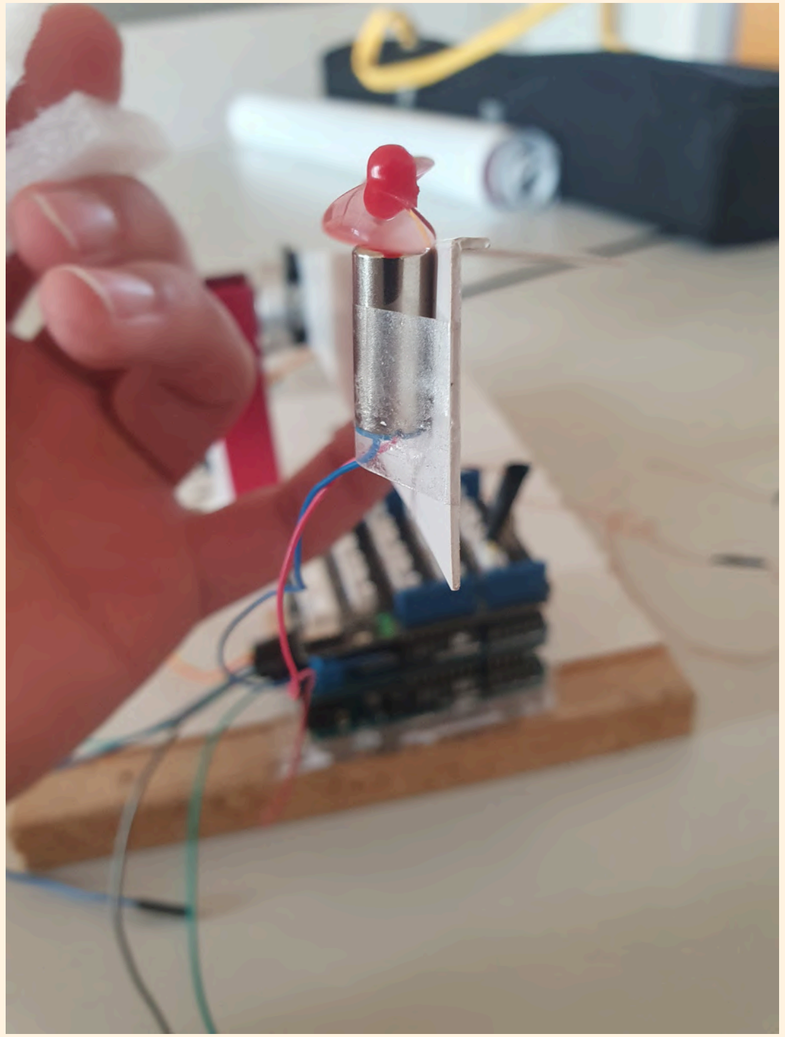



Source : GeeksForGeeks

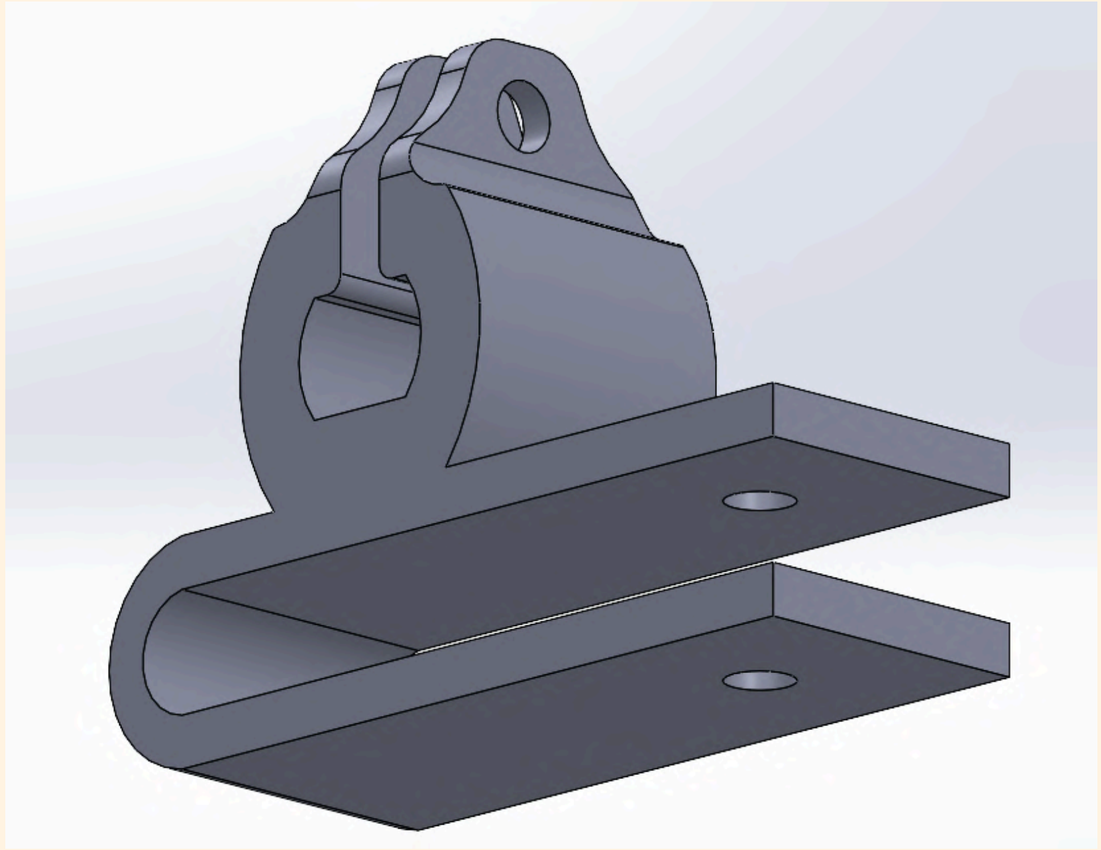
# Annexe : La maquette

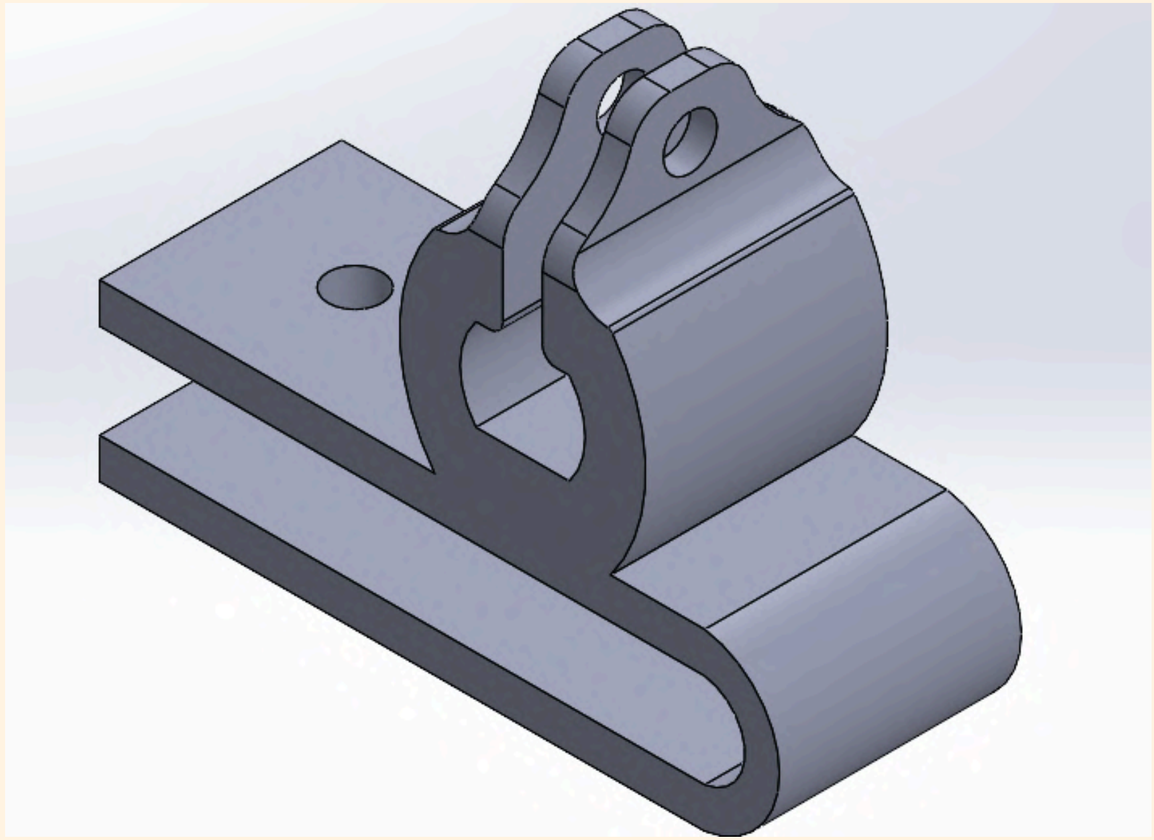






# Annexe : La maquette





# Annexe : Calcul de l’angle brut et atan2

```
////////////////////////////////////// Calcul de l'angle ////////////////////////////////////
float ax = gyro.readFloatAccelX();                  // Lecture des données de l'accéléromètre sur les axes X, Y, Z
float ay = gyro.readFloatAccelY();
float az = gyro.readFloatAccelZ();

float angleBrut = atan2(ax, -az) * 180 / PI;              // Calcul de l'angle selon le quadrant
angleBrut = constrain(angleBrut, -90, 90);
```

## La fonction atan2(x,y)

$$\operatorname{atan2}(y,x) = \begin{cases} \varphi \cdot \operatorname{sgn}(y) & x > 0 \\ \frac{\pi}{2} \cdot \operatorname{sgn}(y) & x = 0 \\ (\pi - \varphi) \cdot \operatorname{sgn}(y) & x < 0 \end{cases}$$

Ou $\varphi$ est l’angle compris entre 0 et pi/2 avec : $\tan(\varphi) = \left|\frac{y}{x}\right|$

Source :Polytech Nice

# Annexe : Le code Arduino (C++)

```cpp
/* Par Bilal BELKHAOUEL CPGE TSI n°SCEI : 11991 */
#include <Wire.h>
#include <LSM6DS3.h>

LSM6DS3 gyro(I2C_MODE, 0x6A);

bool directionState;

int directionPin1 = 12;                 // Moteur gauche
int pwmPin1 = 3;
int brakePin1 = 9;

int directionPin2 = 13;                 // Moteur droit
int pwmPin2 = 11;
int brakePin2 = 8;
```

# Annexe : Le code Arduino (C++)

```
16
17  float angleFiltre;
18  float angleFiltre_precedent = 0;
19
20  int tau = 280;
21
22  float erreur ;
23  float erreur_precedente = 0;
24  /* Par Bilal BELKHAOUEL CPGE TSI n°SCEI : 11991 */
25  float consigne = 0.0;                // Drone à plat
```

```
27  float a = 2.417;
28  float taud = 0.215;
29  float K = 0.642426;
30
31  float commande;
32  float commande_precedente = 0;
33
34  float delai = 10;
35  float deltaT = 2.39 + delai;                    // Temps entre 2 mesure
```

# Annexe : Le code Arduino (C++)

```
37  int vitesseBase = 130;
38
39  void setup() {
40  Serial.begin(9600);
41
42  if (gyro.begin() != 0) {                          // Initialisation du capteur
43    Serial.println("Erreur de connexion.");
44  }
45  else {
46    Serial.println("Connecté.");
47  }
48  pinMode(directionPin1, OUTPUT);
49  pinMode(pwmPin1, OUTPUT);
50  pinMode(brakePin1, OUTPUT);
51
52  pinMode(directionPin2, OUTPUT);
53  pinMode(pwmPin2, OUTPUT);
54  pinMode(brakePin2, OUTPUT);
55  }
```

# Annexe : Le code Arduino (C++)

```
63  void loop() {
64  digitalWrite(directionPin1, LOW);          // Sens de rotation des moteurs
65  digitalWrite(directionPin2, HIGH);
66  digitalWrite(brakePin1, LOW);             // Moteur bloqué ou non
67  digitalWrite(brakePin2, LOW);
68
69
70  ////////////////////////////////////// Calcul de l'angle ////////////////////////////////////
71  float ax = gyro.readFloatAccelX();                // Lecture des données de l'accéléromètre sur les axes X, Y, Z
72  float ay = gyro.readFloatAccelY();
73  float az = gyro.readFloatAccelZ();
74
75  float angleBrut = atan2(ax, sqrt(ay * ay + az * az)) * 180 / PI;            // Calcul de l'angle selon le quadrant
76  angleBrut = constrain(angleBrut, -90, 90);
```

# Annexe : Le code Arduino (C++)

```
////////////////////////////////////// Filtrage de l'angle ////////////////////////////////////

angleFiltre = (angleBrut + angleFiltre_precedent*(tau/deltaT))/((tau/deltaT) + 1);          //Filtrage de l'angle brut
angleFiltre_precedent = angleFiltre;

////////////////////////////////////// Asservissement ////////////////////////////////////
///* Par Bilal BELKHAOUEL CPGE TSI n°SCEI : 11991 */
erreur = consigne - angleFiltre;
erreur_precedente = erreur;

commande = (K*((deltaT*0.001 + a*taud)*erreur - a*taud*erreur_precedente) + taud*commande_precedente)/(deltaT*0.001 + taud);
commande_precedente = commande;
```

# Annexe : Le code Arduino (C++)

```
94  int moteurDroit = vitesseBase + commande;                    ///* Par Bilal BELKHAOUEL CPGE TSI n°SCEI : 11991 *
95  int moteurGauche = vitesseBase - commande;
96
97  moteurDroit = constrain(moteurDroit, 0, 255);
98  moteurGauche = constrain(moteurGauche, 0, 255);
99
00  analogWrite(pwmPin1, moteurDroit);
01  analogWrite(pwmPin2, moteurGauche);
02
03  Serial.print(angleBrut);                                      // Affichage de l'angle brut et l'angle filtré
04  Serial.print(",");
05  Serial.println(angleFiltre);
06
07  delay(delai);
08
```

# Annexe : Mesure angulaire (Python)

```python
import serial
import csv

port = "COM3"
baud = 9600

csv_file = open(r"C:\Users\bilal\OneDrive\Bureau\CPGE TSI\TIPE\Angles\angles.csv", "w", newline="")
csv_writer = csv.writer(csv_file)
csv_writer.writerow(["Temps", "Angle brut", "Angle filtre"])

ser = serial.Serial(port, baud)
try:
    t = 0
```

# Annexe : Mesure angulaire (Python)

```
    while True:
        line = ser.readline().decode().strip()
        if line:
            parts = line.split(",")
            if len(parts) == 2:
                try:
                    angle_brut = float(parts[0])
                    angle_filtre = float(parts[1])
                    csv_writer.writerow([t, angle_brut, angle_filtre])
                    print(f"{t}, {angle_brut}, {angle_filtre}")
                    t += 1

except KeyboardInterrupt:
    print("Fin de l'enregistrement")
    csv_file.close()
    ser.close()
```

# Annexe : Tracer de diagramme de Bode (Python)

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

a = 5
tau_d = 1
w_m = 1/(np.sqrt(a)*tau_d)
phi_m = np.arcsin((a-1)/(a+1))*(180/np.pi)
numerateur = [a*tau_d, 1]
denominateur = [tau_d, 1]
systeme = signal.TransferFunction(numerateur, denominateur)

frequences = np.logspace(-2.5, 2, 1000)
```

# Annexe : Tracer de diagramme de Bode (Python)

```python
def afficher_gain():
    _, gain_db, _ = signal.bode(systeme, frequences)

    plt.figure(figsize=(8, 4))
    plt.semilogx(frequences, gain_db, 'b', linewidth=2, label = "Réel")
    plt.title("Titre")
    plt.xlabel("Pulsation (rad/s)")
    plt.ylabel("Gain (dB)")
    plt.grid(which='both', linestyle=':')
    plt.tight_layout()
    plt.legend()
    plt.show()
```

# Annexe : Tracer de diagramme de Bode (Python)

```python
def afficher_phase():
    _, _, phase_deg = signal.bode(systeme, frequences)

    plt.figure(figsize=(8, 4))
    plt.semilogx(frequences, phase_deg, 'b', linewidth=2, label = "Réel")
    plt.axvline(w_m, color='g', linestyle=':', label=f'$ω_m = {w_m:.2f}$ rad/s')
    plt.axhline(phi_m, color='r', linestyle=':', label=f'$φ_m = {phi_m:.2f}$°')
    plt.title("Titre")
    plt.xlabel("Pulsation (rad/s)")
    plt.ylabel("Phase (°)")
    plt.grid(which='both', linestyle=':')
    plt.tight_layout()
    plt.legend()
    plt.show()
```

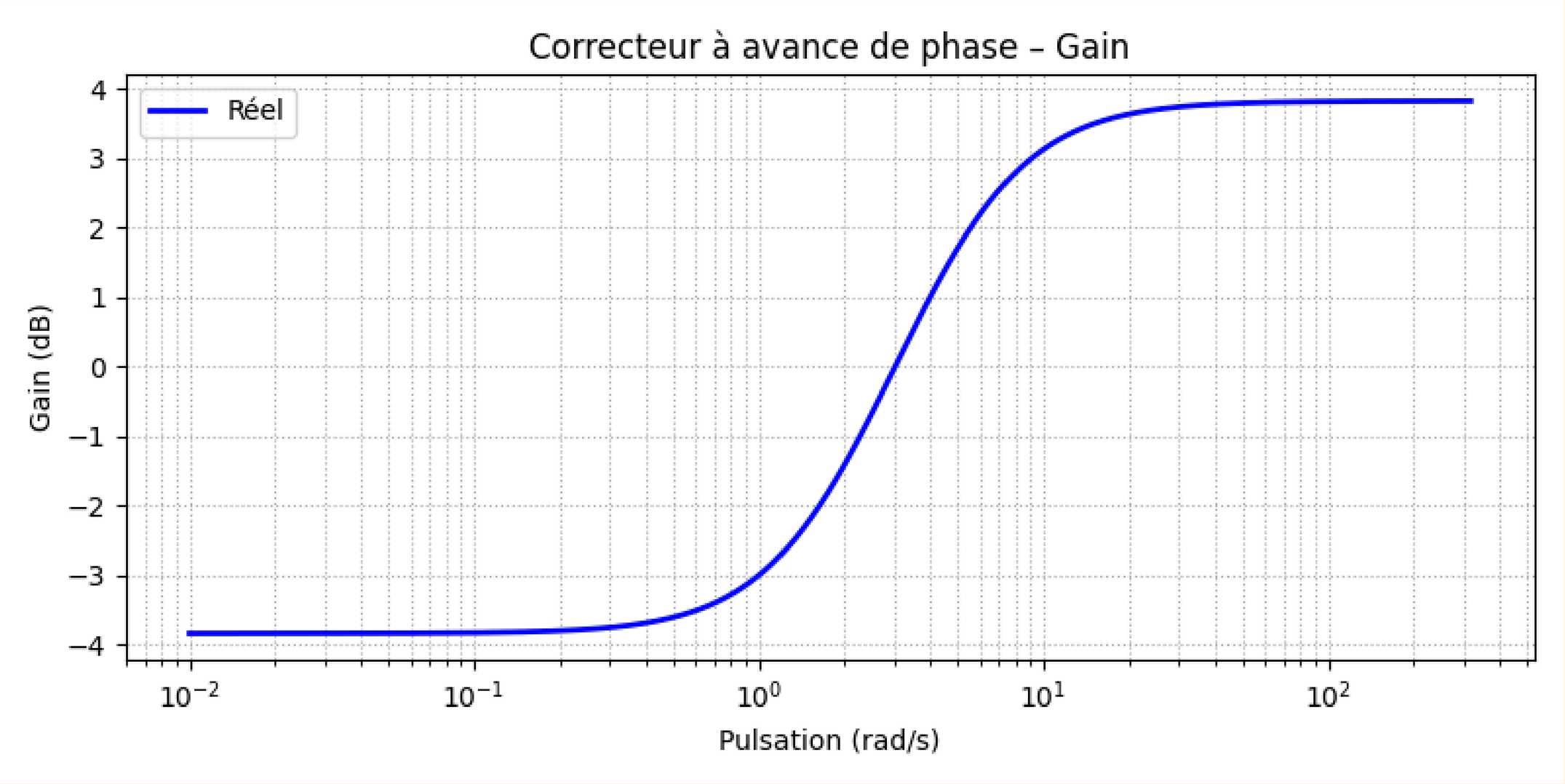
# Annexe : Temps d’échentillonage

```
7    te = micros();
8    //// Calcul ////
9    ts = micros();
10
11   deltaT = ts - te;
12
```

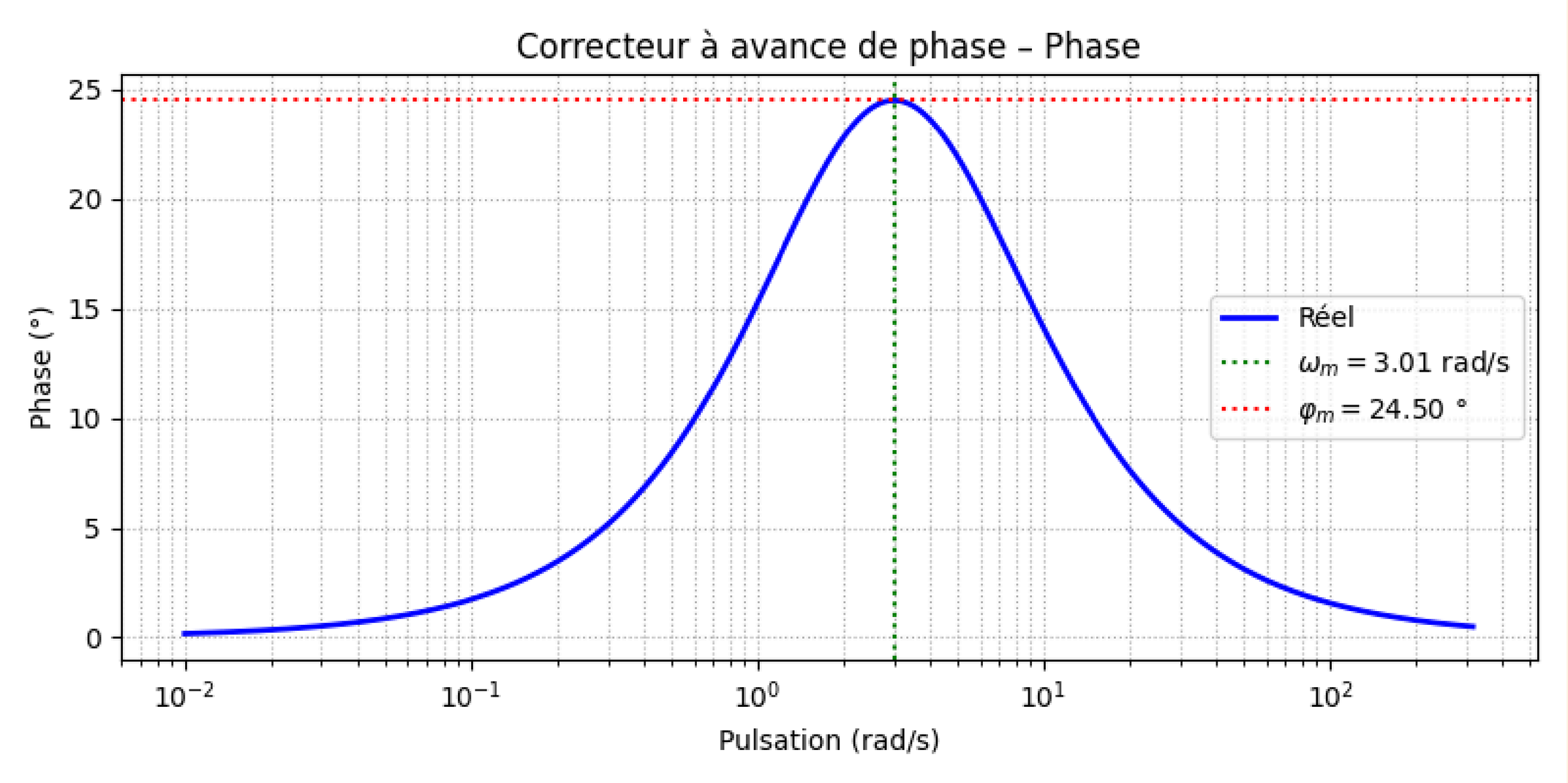
Variation entre 2.35 et 2.45 ms

**De plus, si un 'delay' n'est pas ajouté, il y a une surcharge du processeur au fil du temps, et le temps de mesure entre deux points augmente progressivement.**

# Annexe : Le correcteur à avance de phase



# Annexe : Le correcteur à avance de phase



# Annexe : Les correcteurs

| Correcteur | FT | Stabilité | Précision | Rapidité |
|---|---|---|---|---|
| P | $C(p) = K$ | ↘ si $K > 1$<br>↗ si $K < 1$ | ↗ si $K > 1$<br>↘ si $K < 1$ | ↗ si $K > 1$<br>↘ si $K < 1$ |
| I | $C(p) = \frac{1}{\tau_i\, p}$ | ↘↘ | ↗↗ | ↗ si $\frac{1}{\tau_i} > \omega_{0dB-BO}$<br>↘ si $\frac{1}{\tau_i} < \omega_{0dB-BO}$ |
| D | $C(p) = \tau_d\, p$ | ↗↗ | ↘↘ | ↘ si $\frac{1}{\tau_d} > \omega_{0dB-BO}$<br>↗ si $\frac{1}{\tau_d} < \omega_{0dB-BO}$ |
| PI | $C(p) = K\frac{1+\tau_i\, p}{\tau_i\, p}$ | ↘ si $K > 1$<br>↗ si $K < 1$ | ↗↗ | ↗ si $K > 1$<br>↘ si $K < 1$ |
| PI réel : retard de phase | $C(p) = K\frac{1+\tau\, p}{1+b\,\tau\, p}$<br>$b > 1$ | ↗<br>(une fois réglé) | ↗<br>(une fois réglé) | ↘<br>(une fois réglé) |
| PD réel : avance de phase | $C(p) = K\frac{1+a\,\tau\, p}{1+\tau\, p}$<br>$a > 1$ | ↗<br>(une fois réglé) | ↘<br>(une fois réglé) | ↗<br>(une fois réglé) |

Source : professeur M.Lacombe

# Annexe : illustration et diagramme

Pour créer les diagramme ( Schéma Bloc,  schéma cinématique...) : Draw.io

Illustration non sourcé : Canva.com